



Cite this: DOI: 10.1039/d6dd00273k

CatGo: Bridging CLI coding agents with interactive structure and workflow management for computational chemistry

Guangsheng Liu,^a Xiao Ma,^b Leshen Zhang,^b Jenedith Pascasio,^b Jonathan Yang,^b Yuxiang Chen^c and Wan-Lu Li^{*ab}

Computational chemistry is increasingly limited not by individual calculations but by the manual effort of assembling, parameterizing, executing, and monitoring multi-step workflows across heterogeneous engines and computing environments; existing frameworks ease provenance and job management yet remain code-centric, limiting accessibility. Here we present CatGo, an AI-native computational chemistry workbench operable directly by large-language-model (LLM) coding agents through a Model Context Protocol (MCP) interface. Beyond a workflow engine, CatGo provides an interactive three-dimensional structure builder with atom-level editing, adsorption-site and adsorbate-placement tools, an integrated terminal for direct HPC access, and a bidirectionally synchronized visual editor, all sharing a single state so that human users and agents operate on the same structures and workflows. Researchers state objectives in natural language; agents then construct, validate, execute, and monitor directed-acyclic-graph workflows across VASP, CP2K, ORCA, LAMMPS, and machine-learning interatomic potentials, aided by compact graph-state serialization and batch mutations that cut LLM tool-calling overhead. We demonstrate an end-to-end Pt(111) oxygen-reduction workflow generated from a single natural-language instruction, establishing CatGo as a reproducible, agent-drivable workbench for computational chemistry.

Received 10th May 2026
Accepted 28th July 2026

DOI: 10.1039/d6dd00273k

rsc.li/digitaldiscovery

1 Introduction

Computational materials science and heterogeneous catalysis research increasingly rely on complex multi-step computational workflows involving structure construction, electronic-structure calculations, molecular dynamics simulations, high-performance computing (HPC) execution, and post-processing analysis. Traditionally, these tasks are distributed across fragmented combinations of command-line tools, Python scripts, visualization software, and heterogeneous computational backends, each with distinct interfaces, file formats, and execution models. Widely used tools such as VESTA,² the Atomic Simulation Environment (ASE),³ and Python Materials Genomics (pymatgen)⁴ have significantly advanced structure visualization, manipulation, and atomistic modeling, while newer workflow systems have expanded automation across job submission and analysis pipelines.⁵ In parallel, large language model (LLM) applications in chemistry and catalysis are rapidly

evolving from isolated assistant tools toward workflow-aware scientific systems, including automated literature synthesis,⁶ chemistry-agent planning,^{7,8} autonomous laboratories,⁹ AI-assisted scientific reasoning frameworks,¹⁰ and recent perspectives on AI agents for scientific discovery and energy-materials research.^{11,12} Domain-specific agents have begun to target computational-chemistry and materials workflows directly. For electronic-structure and atomistic-simulation orchestration, recent systems include autonomous VASP simulation (VASPilot),¹³ DFT- and MLIP-driven materials-simulation engines such as DREAMS,¹⁴ GENIUS,¹⁵ and QUASAR,¹⁶ computational-chemistry workflow agents such as ChemGraph,¹⁷ the El Agente family spanning quantum-chemistry, solid-state, structural-editing, tool-generation, and execution-graph orchestration,^{18–24} catalysis-focused autonomous research (CatMaster),²⁵ adsorption-configuration identification (Adsorb-Agent),²⁶ and molecular-dynamics workflow automation (MDCrow).²⁷ For materials discovery and design, agentic systems include LLMatDesign,²⁸ Crystallise,²⁹ metal-organic-framework discovery and generation agents,^{30,31} materials-characterization workflows (SciLink),³² and accelerated-design platforms (AGAPI-Agents).³³ Broader LLM-based chemistry assistants and tool- or reasoning-augmented agents include Aitomia,³⁴ ChemToolAgent,³⁵ ChemAgent,³⁶ ChemAmp (formerly ChemHAS),³⁷ MT-Mol,³⁸ and hypothesis-discovery

^aProgram of Materials Science and Engineering, University of California San Diego, La Jolla, CA 92093, USA. E-mail: wal019@ucsd.edu^bAiiso Yufeng Li Family Department of Chemical and Nano Engineering, University of California San Diego, La Jolla, CA 92093, USA^cDepartment of Chemistry, University of Liverpool, L69 7ZD, UK

† These authors contributed equally to this work.

systems (MOOSE-Chem).³⁹ These systems typically expose a fixed, code-defined task repertoire or a single-agent tool-calling loop rather than an interactively constructed, agent-extensible workflow graph synchronized with a live visual editor; where a frontend is present at all, it is generally a chat console or a read-only visualization dashboard, and none couples it to an interactive graphical structure-building environment (direct surface cutting, adsorbate placement, solvation, and doping in a 3D editor) of the kind CatGo provides. Together, these developments point toward a broader transition from fragmented computational toolchains to integrated, AI-assisted scientific infrastructure.

Despite these advances, existing computational chemistry workflow frameworks, including FireWorks,⁴⁰ AiiDA,^{41,42} jobflow,⁴³ atomate2,⁴³ and quacc,⁴⁴ remain fundamentally code-centric. Workflow construction typically requires users to define Python scripts, job specifications, or explicit dataflow graphs, creating substantial barriers for experimental researchers and introducing friction even for expert computational users during rapid workflow iteration. At the same time, interoperability between structure editing, heterogeneous compute resources, workflow orchestration, and AI-agent control remains limited. Current systems generally lack a unified execution-grade framework in which LLM agents, workflow graphs, structure editors, computational backends, and provenance-aware analysis operate within a shared and continuously synchronized environment. As a result, much of the workflow state and execution logic still resides implicitly in manual scripting and user intervention rather than in a reproducible and agent-operable scientific system.

Here, we present CatGo, a unified open-source computational-chemistry workbench that addresses these limitations through an AI-native architecture. Rather than a workflow tool alone, CatGo integrates, as components of a single synchronized environment: (1) a high-performance 3D structure environment for crystals, molecules, and surfaces, built upon the open-source matterviz library⁴⁵ and extended with catalysis-oriented structure construction, adsorbate placement, and workflow-coupling capabilities; (2) a comprehensive structure-generation toolkit supporting slabs, supercells, adsorbates, interfaces, advanced structural models, and atom-level editing; (3) multi-engine electronic-structure workflow support with fully integrated execution pipelines for VASP,^{46,47} CP2K,^{48,49} ORCA,⁵⁰ and LAMMPS,⁵¹ together with declarative task-node interfaces for Quantum ESPRESSO,^{52,53} Gaussian, Q-Chem, and GROMACS, for which executor integration is currently under development; (4) a visual node-based workflow editor for constructing, monitoring, and executing multi-step computational pipelines; (5) an integrated terminal providing direct access to the HPC file system, input editing, and job control; and (6) an LLM-driven AI assistant capable of interacting with the structure editor, terminal, and workflow engine through natural-language instructions. Unlike conventional code-first workflow frameworks, CatGo enables workflow construction through natural-language interaction with coding-capable agents such as Claude Code,⁵⁴ Codex CLI,⁵⁵ and Gemini CLI,⁵⁶ operating through a Model Context Protocol (MCP)⁵⁷ tool

interface. In this article, we describe the architecture, workflow model, and AI-agent integration of CatGo, and demonstrate its functionality through automated end-to-end computational catalysis workflows, including a natural-language-generated Pt(111) oxygen reduction reaction workflow and a numerical reproduction of the multi-facet Ni catalyst study reported by Kreitz *et al.*¹

2 Design philosophy and architecture

CatGo is motivated by a systems-level challenge in contemporary computational catalysis research: although mature software exists for many individual tasks, practical workflows remain distributed across heterogeneous interfaces, file formats, execution environments, and computational backends. As a result, researchers frequently rely on ad hoc scripting, manual file transfer, and repeated format conversion to connect different stages of the computational pipeline. In many cases, the workflow logic and execution state remain implicitly managed by the user rather than represented as an explicit and operable system object. CatGo addresses this challenge by treating the workflow itself, rather than the individual calculation or visualization step, as the central unit of platform design.

As shown in Fig. 1, the architecture of CatGo is organized around three core design principles. First, the platform adopts a unified data model that minimizes serialization boundaries between structure generation, visualization, analysis, and workflow execution. Second, workflow nodes are defined in terms of scientific tasks rather than software-specific procedures, enabling the same workflow representation to operate across heterogeneous electronic-structure engines. Third, the platform exposes this shared runtime environment through a common programmable interface, allowing AI-assisted workflow control to function as an intrinsic component of the architecture. Together, these principles establish a layered workflow framework in which the unified data model maintains a shared runtime state across modules, task abstraction enables backend portability, and the programmable interface provides consistent access for both human users and language-model agents.

2.1 Comparison with existing workflow frameworks

Existing workflow frameworks for computational chemistry, including FireWorks,⁴⁰ AiiDA,^{41,42} jobflow together with its atomate2 ecosystem of reusable workflows,⁴³ and the quacc workflow layer built upon covalent and ASE,⁴⁴ have substantially advanced provenance tracking, workflow scheduling, restart management, and distributed computational execution. These platforms provide robust infrastructures for constructing and managing complex multi-step computational pipelines through programmable workflow definitions and modular execution environments. In parallel, command-line utilities such as AiiDA's *verdi* and FireWorks' *lpad* have further streamlined workflow orchestration within high-performance computing environments.

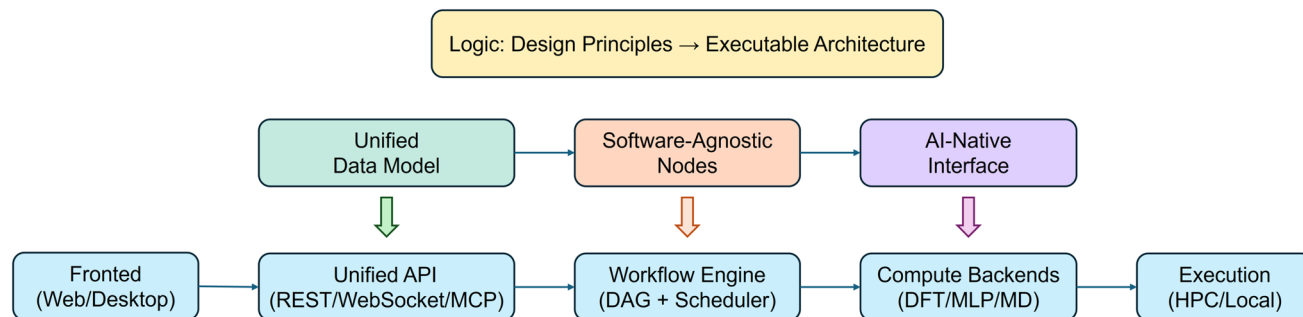


Fig. 1 Logical architecture of CatGo. Top row: three design principles (Unified Data Model, Software-Agnostic Nodes, AI-Native Interface). Bottom row: the five-layer execution stack (Frontend, Unified API, Workflow Engine, Compute Backends, Execution). Colored vertical arrows map each principle onto the stack layer that realizes it.

CatGo is designed as a complementary AI-native computational-chemistry *workbench* rather than a workflow framework alone: the workflow engine is one component of an integrated environment that also provides interactive three-dimensional structure construction and atom-level editing, an integrated terminal for direct HPC file-system access and job control, and a natural-language agent interface, all operating on a shared, continuously synchronized state. In CatGo, natural language serves as the primary human interaction layer, while machine-level operations are exposed through a MCP tool interface. Both interfaces interact with the same underlying workflow graph and structure state, which is synchronized in real time with an interactive Svelte-based visual editor. Rather than replacing existing workflow ecosystems, CatGo inter-operates with established computational chemistry tools such as ASE and pymatgen and interfaces directly with heterogeneous HPC scheduling environments. The platform therefore functions not as an additional workflow layer alone, but as a unified workbench that integrates conversational AI interaction, interactive structure building and editing, visual workflow construction, an integrated terminal, and heterogeneous computational execution within a single continuously synchronized environment.

What distinguishes CatGo from these frameworks is therefore not the underlying execution infrastructure (multi-engine dispatch, provenance tracking, and HPC scheduling are shared capabilities) but that CatGo is an integrated *workbench* rather than a workflow framework alone. As summarized in Table 1, natural-language workflow construction, an agent-operable MCP interface, a bidirectionally synchronized drag-to-connect visual editor, an integrated three-dimensional environment for structure construction and atom-level editing, and an integrated terminal for direct HPC file-system access and job control are, to our knowledge, not jointly provided by FireWorks, AiiDA, atomate2, or quacc, whose workflows are defined programmatically. CatGo is interoperable rather than competing: it imports atomate2 and quacc workflow templates and builds on ASE and pymatgen, so it extends the existing ecosystem with an agent- and human-facing construction layer rather than replacing the underlying engines. Two capabilities in particular set CatGo apart: the AI assistant can define new

task nodes at runtime, including methods absent from the predefined palette, so the available node set is agent-extensible rather than fixed; and an alternative file-first *campaign* mode drives multi-stage studies from a human-readable folder-and-markdown tree whose names are directly recognizable to the user, avoiding the database-backed, opaquely identified state typical of conventional workflow engines.

2.2 Unified data model

Contemporary computational catalysis workflows commonly involve multiple software environments spanning structure generation, electronic-structure calculations, workflow execution, visualization, and post-processing analysis. Structure preparation may be performed using ASE,³ pymatgen,⁴ or related scripting libraries; input generation may involve VASP-kit,⁵⁸ custom scripts, or manual editing; calculations are executed on heterogeneous HPC resources; and downstream analysis often relies on additional visualization and analysis tools such as VESTA,² Jmol,⁵⁹ LOBSTER,⁶⁰ and matplotlib-based workflows.⁶¹ As a result, workflow state, intermediate data, and execution logic are frequently distributed across multiple disconnected environments.

CatGo adopts a unified data model in which the structure object serves as the central runtime representation shared across all platform modules. Structure generation, visualization, adsorption-site analysis, workflow execution, and electronic-structure analysis operate directly on the same object representation rather than through intermediate file exchange. Structural updates therefore propagate continuously across the platform: generated slabs become immediately available for adsorption-site analysis, adsorbate placement is synchronized with the visualization environment, and updated configurations can be passed directly into downstream workflow nodes. The same framework extends to HPC interaction, enabling remote file-system access, input editing, workflow execution, and output inspection within a single interface layer.

The unified data model further provides the foundation for the directed acyclic graph (DAG) workflow system described in Section 5. Because workflow components share a common representation and interface layer, computational tasks can be composed as graph nodes connected through typed data edges,

Table 1 Comparison of CatGo with established computational-chemistry workflow frameworks. CatGo is an integrated workbench rather than a workflow framework alone, so the comparison spans interactive and agent-facing capabilities in addition to workflow construction. ✓ = provided; – = not provided; lim. = limited or indirect. Execution-level capabilities (multi-engine dispatch, provenance, HPC scheduling) are shared across all frameworks

Capability	FireWorks	AiiDA	atomate2	Quacc	CatGo
Programmatic workflow definition	✓	✓	✓	✓	✓
Multi-engine execution	✓	✓	✓	✓	✓
Provenance tracking	✓	✓	✓	✓	✓
HPC scheduler integration	✓	✓	✓	✓	✓
Natural-language workflow construction	—	—	—	—	✓
Agent-operable interface (MCP)	—	—	—	—	✓
Interactive drag-to-connect editor	—	Lim	—	—	✓
Bidirectional GUI-state synchronization	—	—	—	—	✓
Integrated 3D structure/adsorbate environment	—	—	—	—	✓
Interactive structure construction & atom-level editing	—	—	—	—	✓
Integrated terminal/HPC file access	—	—	—	—	✓
Agent-defined/runtime-created nodes	—	—	—	—	✓
Human-readable file-first orchestration	—	—	—	—	✓
Import of atomate2/quacc templates	—	—	n/a	n/a	✓

enabling consistent workflow construction, execution, and analysis within a synchronized workflow environment.

2.3 Software-agnostic task abstraction

Computational chemistry workflows increasingly span multiple electronic-structure engines and machine-learning interatomic potential (MLIP) frameworks. Widely used quantum chemistry and atomistic simulation packages include VASP,^{46,47} CP2K,^{48,49} ORCA,^{50,62} Gaussian,⁶³ xTB,⁶⁴ and LAMMPS,⁵¹ while modern MLIP frameworks such as MACE,⁶⁵ CHGNet,⁶⁶ M3GNet,⁶⁷ and SevenNet⁶⁸ are increasingly trained on large-scale catalysis and materials datasets including OC20,⁶⁹ OC22,⁷⁰ and GNoME.⁷¹ As computational workflows become increasingly heterogeneous, interoperability across different execution engines becomes an important architectural consideration.

CatGo addresses this through a software-agnostic task abstraction in which workflow nodes represent scientific operations rather than software-specific procedures. For example, a `geo_opt` node represents a geometry optimization task independent of the backend implementation, while the underlying execution engine (*e.g.* VASP, CP2K, xTB, or an MLIP framework) is specified through a `software` parameter resolved by the workflow dispatch layer at runtime. This abstraction enables workflow portability across computational engines while preserving a consistent workflow representation. Consequently, workflows can combine heterogeneous execution layers within the same DAG, such as MLIP-based pre-relaxation followed by DFT refinement, without modifying the workflow topology itself.

The same abstraction further enables unified orchestration across heterogeneous computational resources. Lightweight structure-processing tasks can execute locally or within browser-side environments, MLIP-based relaxations can be dispatched to local CPU/GPU resources, and first-principles calculations can be submitted directly to remote HPC

schedulers, while maintaining a shared workflow state for monitoring and data management. The full workflow-node library spanning input generation, atomistic modeling, electronic-structure calculations, analysis, and control-flow operations is summarized in Section 5.2.

At the time of writing, the computational backends supported by CatGo differ in their degree of integration within the workflow engine. VASP, CP2K, ORCA, LAMMPS, and ASE-based MLIP frameworks including MACE, CHGNet, M3GNet, UMA-S *via fairchem*,⁷² xTB, and Sella⁷³ are implemented as fully integrated execution backends supporting automated input generation, scheduler dispatch, live execution monitoring, and output parsing within the unified workflow environment. Gaussian, Q-Chem,⁷⁴ Quantum ESPRESSO, GROMACS, and AMBER are currently implemented as declarative task-node interfaces that support structured input generation and workflow-level integration, while automated scheduler dispatch and output-parsing functionality remain under development. This distinction is indicated by a dagger (†) in Table 2. All end-to-end workflow demonstrations discussed in Section 7 and Section S3.6 employ fully integrated execution backends.

2.4 Extensibility: custom nodes across three workflow modes

CatGo is not limited to a fixed set of predefined task nodes. New methods are added without modifying core code, and because CatGo supports three workflow modes in which the node library plays a different role, a fixed node set is not a hard constraint.

2.4.1 Visual node-based workflows. In the drag-to-connect editor the workflow is built from nodes, and a new method enters in one of two ways. To add a new compute backend for an existing calculation type, one writes a declarative engine definition that maps the abstract calculation types onto the package's parameters, input templates, run command, and output parsing; the new backend then appears automatically as

Table 2 Node library of the CatGo workflow system. Unified calculation nodes accept a `software` parameter to select the backend. Fully integrated executors (input generation, scheduler dispatch, and output parsing) are available for VASP, CP2K, ORCA, LAMMPS, and the MLP/xTB/Sella ASE-based backends; entries marked with a dagger (†) are currently exposed as declarative task-node interfaces whose full executor integration is in development

Node type	Backends/description
Input	
<code>structure_input</code>	File, database, or upstream
<code>struct_list_in</code>	Batch of structures
Calculation	
<code>geo_opt</code>	VASP, CP2K, ORCA, xTB, MLP; Gaussian [†]
<code>single_point</code>	VASP, CP2K, ORCA, xTB, MLP; Gaussian [†]
<code>cell_opt</code>	VASP, CP2K
<code>md</code>	VASP, CP2K, LAMMPS, MLP; GROMACS [†] , AMBER [†]
<code>Freq</code>	VASP, CP2K, ORCA, MLP; Gaussian [†]
<code>ts_search</code>	Sella, ORCA NEB-TS, MLP-NEB
<code>Irc</code>	ORCA
Build/transform	
<code>slab_gen</code>	Surface slab construction
<code>adsorbate_place</code>	Place adsorbate at site
<code>batch_adsorbate_place</code>	Batch multi-adsorbate
<code>batch_generate</code>	Combinatorial generation
<code>doping_gen</code>	Elemental substitution
Analysis	
<code>dos_analysis</code>	Density of states
<code>cohph_analysis</code>	COHP/ICOHP
<code>free_energy</code>	ΔG with ZPE, $T\Delta S$
<code>charge_analysis</code>	Bader charge ⁷⁵
<code>conv_check</code>	Energy/force convergence
<code>energy_compare</code>	Energy ranking
<code>pick_best</code>	Select optimal structure
<code>export_data</code>	CSV/JSON export
Logic/control	
<code>condition</code>	Conditional branching
<code>loop</code>	Iterative refinement

Table 2 (Contd.)

Node type	Backends/description
<code>merge</code>	Merge parallel branches
<code>map</code>	Fan-out (parallel subgraph)
<code>aggregate</code>	Fan-in (collect and rank)

a `software` option on the corresponding node, with no frontend code. To add a genuinely new node type, one registers a workflow-node plugin or an audited, sandbox-executed AI-built tool whose node definition is read by the frontend and rendered directly in the node palette, again without frontend code. The shipped engine library already spans VASP, CP2K, ORCA, LAMMPS, xTB, and machine-learning-potential backends, so a package such as NWChem or ADF is added as an analogous engine definition.

2.4.2 Agent-driven backend workflows. The same backend workflow engine can be built and executed purely by conversing with an AI agent through the MCP interface, with no visual editor required, and a single batched call can add many nodes and edges at once. At the engine layer this mode is software-agnostic: nodes are generic scientific operations (geometry optimization, single point, frequency, and so on), and the engine that executes each one is a `software` parameter resolved through a pluggable engine registry, so adding support for a new package amounts to registering a generator and a collector. The agent-built workflow shares the same graph state as the visual editor and can therefore be visualized as a node graph at the user's option, and the pre-submission input-file review (the human-in-the-loop check of the generated inputs) applies equally.

2.4.3 Markdown-driven campaigns. The most flexible mode dispenses with predefined nodes entirely: a multi-stage study is driven from a human-readable folder-and-markdown tree (the file-first campaign mode). The agent reads the markdown, renders the inputs, submits through standard `ssh/sbatch`, and writes status back to the tree, so the mode is naturally agnostic to the underlying software and cluster, and the user's role is to confirm the inputs. Extensibility here operates at the level of the whole study rather than the individual node.

The three modes are complementary rather than mutually exclusive, and all are operable by an AI agent (Table 3). The visual editor is the most legible: every node parameter carries inline help shown on hover which, together with the live graph, makes it well suited to teaching and to reproducible, fixed pipelines. The agent-driven backend mode targets programmatic, high-throughput construction and interoperates with existing ecosystems by importing `atomate2` and `quacc` templates. The markdown campaign mode is the most flexible. Because orchestration is carried by an agent that reads and writes a plain-text tree rather than by a fixed graph, it

Table 3 The three workflow modes of CatGo and their complementary roles. All three are driveable by an AI agent, and the two database-backed modes share a single graph representation; they differ in how a study is constructed, executed, monitored, and recorded, and therefore in the kind of research each best supports

	Visual node editor	Agent-driven backend	Markdown campaign
Construction	Drag-to-connect node graph in the browser	The agent assembles the same graph through the MCP interface	A human-readable folder-and-markdown tree
Execution	Shared database-backed DAG engine	Shared database-backed DAG engine	The agent submits jobs directly over SSH; no database
Visualization	Native and live	Optional, on demand	Folder and markdown index
Control flow/iteration	Structured and bounded: parallel fan-out and convergence loops over fixed criteria, defined at construction	The same engine and the same structured control flow	Open-ended: the agent diagnoses failures, revises parameters and the plan, and logs each intervention
Failure handling	Transient execution errors are retried; the running graph is not revised	As the visual editor	The agent re-plans in response to unanticipated scientific outcomes
Monitoring	Fixed live status streaming	Fixed live status streaming	Flexible: a scheduled agent can track arbitrary user-defined metrics and act on them
Provenance	Queryable database run records	Queryable database run records	File-first and human-readable (status, result, lessons, and plan files plus a navigable tree); portable and version-controllable
Extension	New backend <i>via</i> a declarative engine definition, or a new node type <i>via</i> a plugin or AI-built tool, with no frontend code	The same mechanisms; imports atomate2 and quacc templates	Add a skill as a markdown document; coverage grows with the skill library
Best suited for	Teaching, onboarding, and reproducible fixed pipelines; every parameter carries inline help	Programmatic, high-throughput construction and interoperability with existing ecosystems	Exploratory, high-variance day-to-day research across software and clusters, and long-running monitored studies

accommodates the situations that rigid pipelines handle least well: an unanticipated failure that requires changing parameters or the plan itself, and monitoring that must follow problem-specific metrics. The two database-backed modes retry transient execution errors automatically but do not revise a running graph, whereas the campaign agent re-plans and records each intervention, and a scheduled agent can watch arbitrary metrics and act on them. The corresponding trade-off is that campaign provenance is file-first and human-readable rather than a queryable database; we regard this transparency and portability as an advantage for everyday research.

Structural descriptors extend through the same pluggable mechanism: the analysis nodes are post-processing steps, so alternative hydrogen-bond definitions, water-tetrahedrality measures, and adjacency-environment descriptors (of the kind shown for the MoS₂/water interface in Fig. 5) are added as analyzer plugins or configuration rather than framework changes. For transparency, several declarative backends (Gaussian, Quantum ESPRESSO, Q-Chem, GROMACS) are at present input-generating interfaces whose end-to-end executor integration is in progress.

2.5 Potential-assisted multi-stage optimization

To support practical high-throughput studies, CatGo explicitly supports staged optimization strategies that combine machine-learned potentials with DFT refinement in a single DAG. A common pattern is `geo_opt(software=mlp) →`

`geo_opt(software=vasp) → single_point/analysis`: the first stage rapidly relaxes a large candidate set using MACE/CHGNet/M3GNet,^{65–67} while the second stage performs physics-based refinement on filtered structures. Because both stages share one node abstraction and one data model, switching between potential types or changing the refinement backend requires only parameter edits, not workflow rewiring.

This design turns “potential-function optimization” into a first-class, reproducible workflow capability rather than an ad hoc preprocessing script. In addition to acceleration, it improves methodological clarity: the workflow artifact records exactly which potential generated each pre-relaxed structure, which DFT settings were used for refinement, and which analysis outputs were used for ranking.

2.6 Execution engine

The CatGo execution engine evaluates workflow DAGs through dependency-aware topological scheduling. Nodes whose dependencies have been satisfied are grouped into execution layers and dispatched automatically, while independent nodes within the same layer can execute concurrently. As upstream tasks complete, downstream nodes become eligible for execution according to the workflow dependency graph.

A representative execution order is illustrated below:

Layer 0: `structure_input`

Layer 1: `slab_gen`, `doping_gen` (parallel)

Layer 2: `geo_opt`

Layer 3: `single_point`, `dos_analysis` (parallel)

This layered scheduling model enables concurrent execution of independent workflow branches while preserving dependency consistency across the computational pipeline.

2.7 AI-Native interface

CatGo is designed such that AI-agent interaction operates directly on the same workflow representation used throughout the platform. Because structure generation, workflow construction, execution management, and analysis share a unified data model and interface layer, workflow operations can be exposed through a consistent MCP interface without requiring additional translation between heterogeneous software environments. Consequently, language-model agents interact with the same workflow graph, structure objects, and execution state available within the graphical interface.

To support efficient agent-driven workflow construction, CatGo employs a compact textual representation of the DAG state during MCP interactions. Workflow-modification operations, including node creation, edge construction, parameter updates, and batch graph edits, automatically return a lightweight graph snapshot appended to the tool response. The snapshot consists of a Kahn-ordered node listing together with summarized edge connectivity and compact workflow parameters. Typical 10–30 node workflows serialize to approximately 400–1200 tokens under the `cl100k_base` tokenizer, while the 24-node ORR workflow described in Section 7 requires 805 tokens. This compact representation enables iterative workflow construction while maintaining a continuously updated workflow state within the conversational context.

The workflow snapshot is designed as a human-readable representation optimized for language-model interpretability and token efficiency during interactive sessions. The authoritative workflow graph remains stored within the backend database and can be retrieved in structured form when necessary. This design enables large multi-node workflows to be constructed and modified within a single conversational session while maintaining synchronization between the MCP interface and the graphical workflow environment.

2.8 CLI-agent operation and bidirectional GUI synchronization

CatGo's MCP server exposes a single unified `catgo_workflow` tool with an action discriminator (`create`, `add_node`, `connect`, `set_params`, `batch`, `validate`, `run`, `status`, `step_error`, among others) alongside focused tools for structure editing, DFT-input generation, catalysis-specific site analysis, and visualization control (Table 4). CLI coding agents (Claude Code, Codex CLI, and Gemini CLI) attach to this MCP server through their native configuration, so a user with an existing CLI subscription pays no additional API cost for the AI layer and retains full locality of control. Agent sessions are SDK-backed and resumable: a long authoring session that is interrupted (*e.g.* by a laptop lid-close or a cluster maintenance window) resumes from the last checkpointed state rather than restarting. Critically, the MCP-driven backend edits are

Table 4 Representative MCP workflow tool actions (a subset of the 21 actions exposed by the unified `catgo_workflow` tool) available to AI agents for programmatic workflow construction and execution. In addition to the per-mutation actions listed below, a `batch` action accepts multiple `add_node/connect/set_params` operations in a single call, enabling an AI agent to construct a multi-node screening DAG in one MCP invocation

Action	Description
<code>create</code>	Create a new workflow (auto-adds <code>structure_input</code> node)
<code>add_node</code>	Add a node with type and parameters (defaults auto-merged)
<code>remove_node</code>	Remove a node and its connected edges
<code>connect</code>	Connect two ports (with cycle detection and type validation)
<code>set_params</code>	Incrementally update node parameters
<code>validate</code>	Return errors and warnings for the current graph
<code>Run</code>	Execute the workflow (after validation)
<code>pause/resume</code>	Suspend and resume execution
<code>status</code>	Query per-node execution progress
<code>node_types</code>	List available node types (for AI discovery)
<code>node_details</code>	Return full parameter schema for a node type

propagated to the frontend Svelte workflow editor through a WebSocket reload channel, so a researcher monitoring the GUI sees the agent construct, annotate, and connect nodes in real time. Conversely, graph edits made by hand in the GUI are visible to the agent on its next graph-snapshot read. To our knowledge no prior workflow framework couples a CLI-agent authoring surface with a live-synchronized interactive visual editor.

3 Structure editing, building, and site generation

CatGo provides an integrated structure-construction environment spanning atomistic model generation, interface construction, adsorption-site analysis, and adsorbate placement within a unified workflow framework. Beyond standard slab and supercell preparation, the platform supports advanced structure-generation operations including moiré superlattices,⁷⁶ nanotube construction,⁷⁷ water-layer and pseudo-hydrogen treatments,^{51,78,79} doping, intercalation, atom-level editing, and lattice transformations. This section focuses on two methodological components with direct relevance to catalytic model preparation and high-throughput screening: symmetry-reduced interface registry search and alpha-shape-based adsorption-site identification. Worked examples for elemental substitution (Fe-doped NiOOH(001)) and cation

intercalation (Li into CoO₂) are provided in Section S1 and Section S2, respectively.

3.1 Interface construction and symmetry-reduced sampling

CatGo treats interface construction as a native structure-generation operation within the workflow environment (Fig. 2). The heterostructure builder supports bulk, slab, interfacial, and lateral heterostructure modes through a two-stage search-and-build protocol. During the search stage, ZSL-compatible lattice matches are enumerated under user-configurable tolerances (default: $\text{max_area} = 400 \text{ \AA}^2$, $\text{max_area_ratio_tol} = 0.09$, $\text{max_length_tol} = 0.03$, $\text{max_angle_tol} = 0.01 \text{ rad}$, $\text{max_results} = 50$),^{4,80} and candidate interfaces are ranked according to lattice mismatch and strain

descriptors. During the build stage, selected lattice matches and surface terminations are combined to generate the final interface structure.

Interfacial strain is evaluated from the in-plane deformation gradient $T = F^{-1}S$ and the corresponding symmetric strain tensor $\varepsilon = (T + T^T)/2 - I$, together with the associated Von Mises strain measure,

$$\sqrt{\varepsilon_{11}^2 + \varepsilon_{22}^2 - \varepsilon_{11}\varepsilon_{22} + 3\varepsilon_{12}^2}. \quad (1)$$

For interface-registry analysis, CatGo implements symmetry-reduced lateral translation sampling. The algorithm extracts slab symmetries using SpacegroupAnalyzer,⁴ filters pure in-plane symmetry operations (tolerance 10^{-4}), and determines an

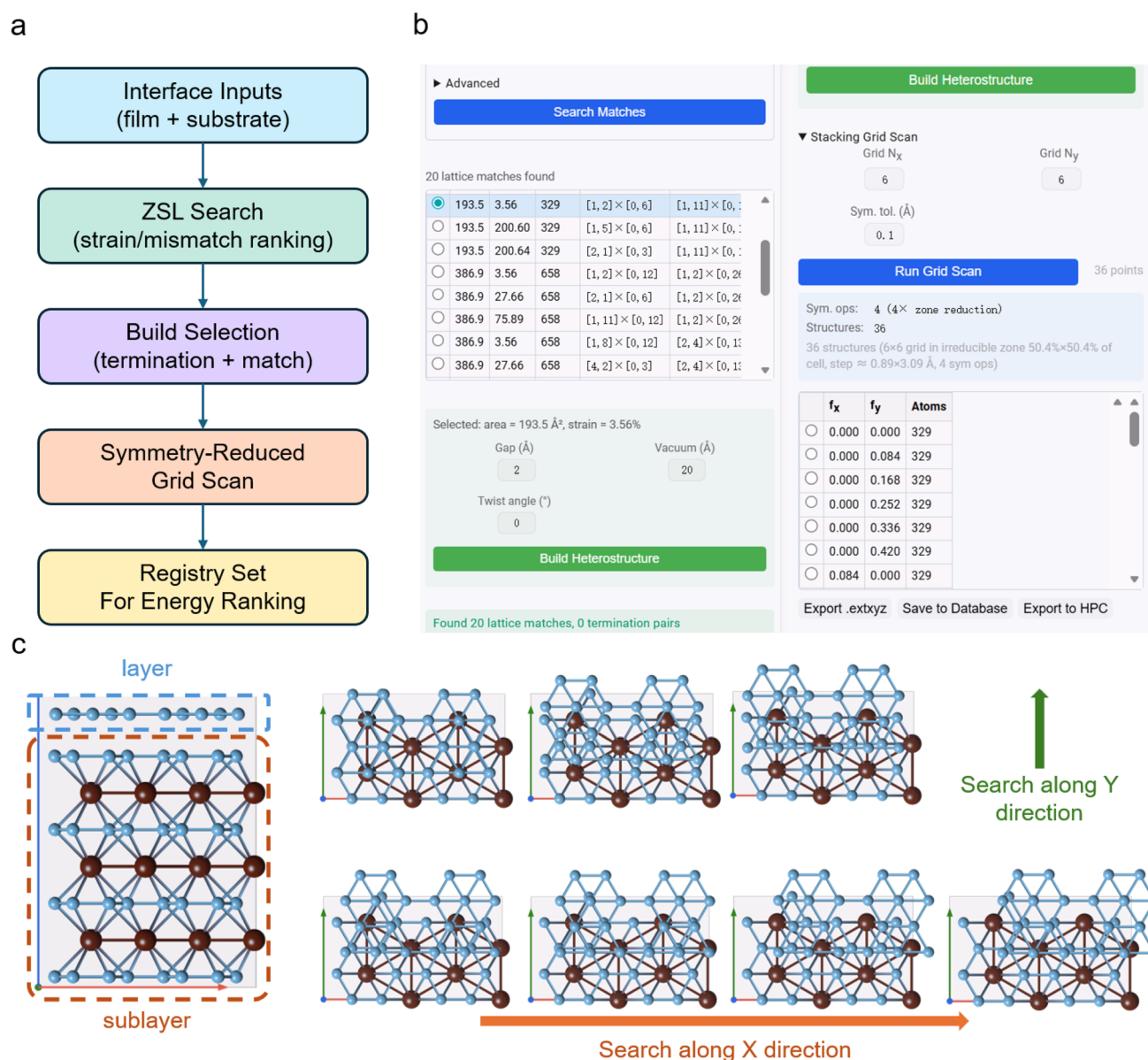


Fig. 2 Interface-centric structure construction. (a) Compact interface workflow logic. (b) Heterostructure search/build evidence panel. (c) Registry-analysis view showing the translation grid and candidate selection workflow.

irreducible translation region through canonical mapping on a dense 120×120 internal grid. User-defined translation grids ($n_x \times n_y$) are then sampled only within this irreducible region. Each sampled configuration is generated by rigid lateral displacement of the film atoms while preserving substrate coordinates and interfacial spacing. For representative interface systems, the resulting symmetry reduction typically decreases the number of sampled registries by approximately one order of magnitude while preserving symmetry-distinct configurations.

3.2 Alpha-shape adsorption site identification and placement

As displayed in Fig. 3, CatGo employs an alpha-shape-based adsorption-site algorithm designed for heterogeneous catalytic structures with non-planar surface geometries. Unlike fixed- z projection approaches, the method can identify adsorption environments on stepped surfaces, concave regions, slab-supported particles, and interfacial pore-like geometries within a unified geometric framework. In slab-supported particle systems, the method further distinguishes terrace, edge, and interfacial adsorption environments relevant to catalytic screening.

The adsorption-site analysis is based on a three-dimensional Delaunay tessellation with in-plane periodic-boundary expansion.^{81–83} Tetrahedra belonging to the alpha complex are retained,⁸⁴ and boundary faces are used to identify exposed atoms and generate candidate top, bridge, and hollow

adsorption sites. The implementation is compiled within the browser-side WASM environment of the interactive structure viewer and can be accessed both through the graphical sites construction panel and through AI-driven structure-editing operations (e.g. `find_adsorption_sites`).

Using an alpha cutoff:

$$R_k < \alpha \text{ (default } \alpha = 2.7 \text{ \AA)}, \quad (2)$$

the method separates exposed and subsurface atoms while preserving local surface curvature and concavity information. Candidate adsorption sites are filtered and returned interactively within the visualization environment, enabling direct inspection during structure preparation.

The adsorption-site workflow is integrated with an automated adsorbate-placement routine (`adsorbate_placement.py`) that aligns user-selected binding atoms with local surface normals, optimizes adsorbate orientation, and removes short-contact geometries prior to workflow export. The resulting slab-adsorbate structures can then be transferred directly into downstream VASP, CP2K, or ORCA workflows.

To support catalytic screening workflows, CatGo provides four adsorbate-definition modes: preset adsorbate libraries, XYZ-based molecular input, PubChem retrieval, and in-view molecular definition. The preset libraries include common intermediates associated with oxygen evolution and reduction reactions (OER/ORR), CO₂ and CO reduction reactions (CO₂RR/CORR), nitrogen reduction reaction (NRR), nitrate reduction

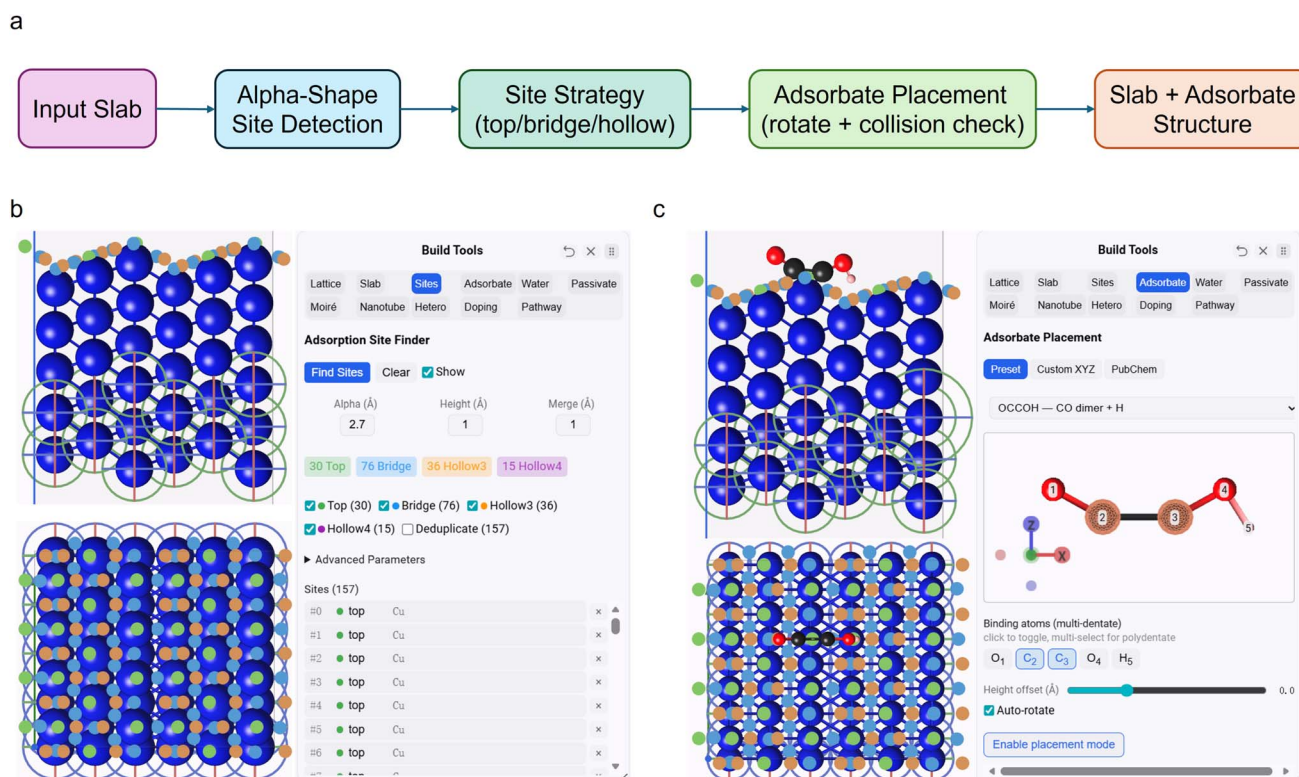


Fig. 3 Integrated adsorbate workflow environment. (a) Workflow logic and data expansion scheme. (b) Adsorption-site selection interface. (c) Adsorbate-definition and placement interface.

reaction (NO_3RR), hydrogen evolution reaction (HER), and related thermocatalytic pathways, enabling reaction-specific adsorbate generation directly within the workflow environment.

4 Analysis capabilities

CatGo provides a broad analysis suite within one shared workflow context, including electronic-structure descriptors, trajectory statistics, interfacial thermodynamics, and free-energy extraction. In this section, we first summarize electronic-structure analysis capabilities and then focus on trajectory and molecular-dynamics analysis as the primary methodological emphasis.

4.1 Electronic structure analysis

CatGo provides a unified electronic-structure visualization and analysis environment supporting outputs from VASP,^{46,47} Quantum ESPRESSO,^{52,53} and related electronic-structure packages. Available analyses include spin-polarized band structures

along user-defined high-symmetry k -paths, projected and total density of states (DOS), crystal orbital Hamilton population (COHP) analysis for bond-strength characterization,^{60,85} interactive Brillouin-zone visualization, and charge-density or molecular-orbital isosurface rendering from Gaussian Cube files. These analyses support interactive energy-range selection, Fermi-level alignment, orbital projection, and direct export for publication-quality visualization.

Importantly, these electronic analyses remain directly connected to the same workflow pipeline used for structure construction and simulation execution. As a result, descriptors such as the d-band center,⁸⁶ ICOHP values, and charge-density distributions can be traced directly back to the corresponding slab construction, adsorption-site generation, and geometry-optimization stages within a reproducible workflow graph.

Fig. 4 demonstrates these capabilities for the $\text{Pt}(111) + \text{O}^*$ system taken from the present ORR workflow: the projected density of states with the Pt d-band center, the slab band structure, the Pt–O crystal orbital Hamilton population, and the

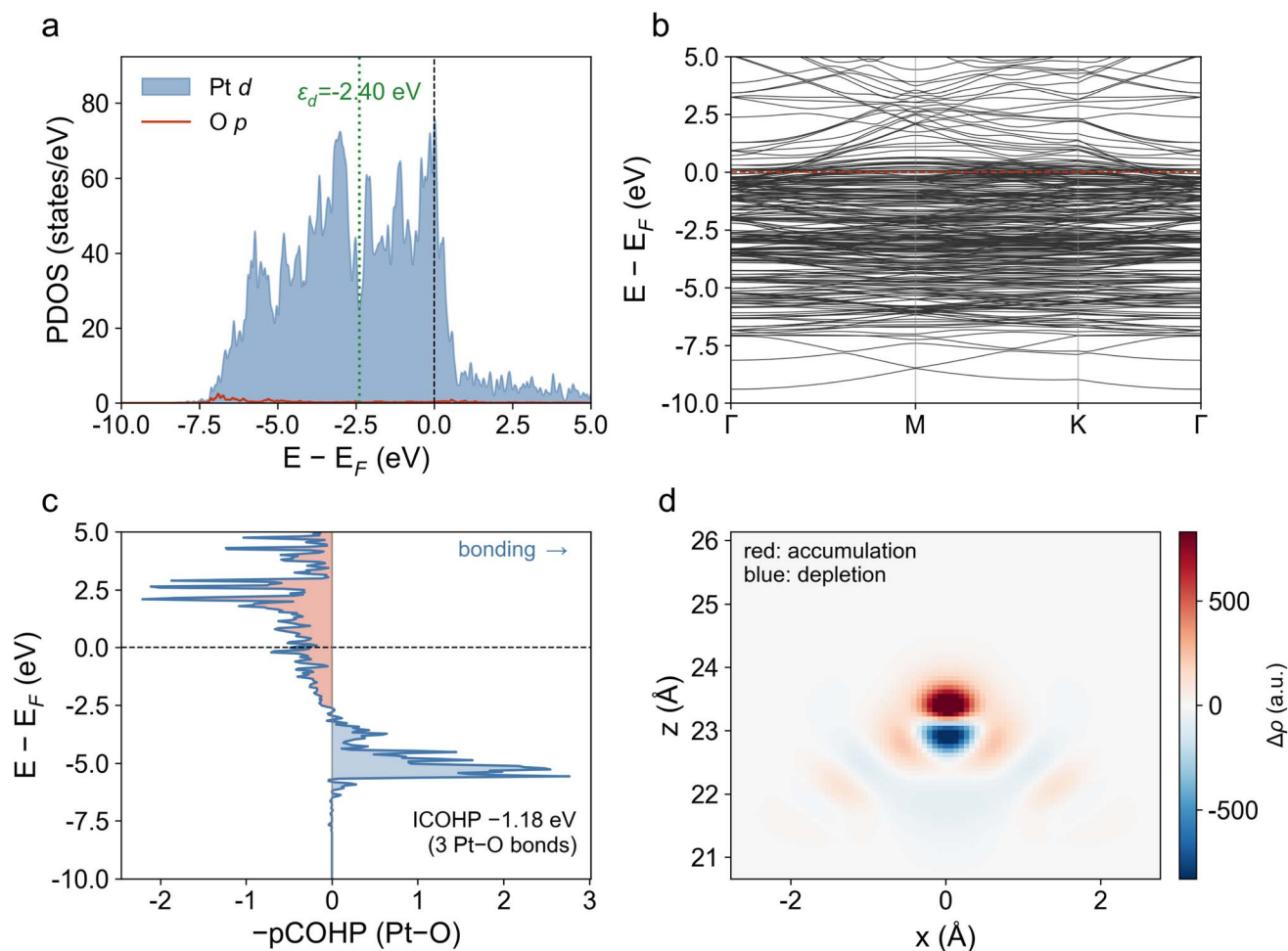


Fig. 4 Electronic-structure analysis of the $\text{Pt}(111) + \text{O}^*$ system, generated by CatGo (VASP + LOBSTER) from the ORR workflow. (a) Projected density of states for Pt d and O p states; the Pt d-band center lies at -2.40 eV. (b) Band structure along Γ -M-K- Γ , reflecting the metallic character of the slab. (c) Crystal orbital Hamilton population ($-\text{pCOHP}$) of the Pt-O bond; the integrated ICOHP is -1.18 eV per bond, averaged over the three fcc Pt-O bonds, with bonding states below the Fermi level. (d) Charge-density difference in a plane through the adsorbed oxygen, showing electron accumulation (red) and depletion (blue) in the O-Pt bonding region.

charge-density difference of the adsorbed oxygen, all generated by the CatGo analysis nodes from the same workflow. These analyses are produced directly within the CatGo frontend rather than by external plotting scripts; full-resolution screenshots of the four analysis views as displayed in the interface are provided in ESI Fig. S10.

4.2 Trajectory and molecular dynamics analysis

CatGo provides a comprehensive trajectory-analysis framework for studying structural dynamics, interfacial solvation, transport phenomena, and free-energy landscapes from molecular dynamics and AIMD simulations. The framework supports analysis of both equilibrium and constrained trajectories, including radial distribution functions, diffusion coefficients, density profiles, hydrogen-bond dynamics, cavitation thermodynamics, and constrained free-energy calculations. Supported trajectory formats include VASP XDATCAR,⁴⁷ Extended XYZ, ASE.traj,³ LAMMPS dump,⁵¹ GROMACS XTC/TRR,⁸⁷ AMBER NetCDF,⁸⁸ HDF5, and PDB formats, with periodic-boundary-condition support and automatic topology inference where applicable.

All trajectory-analysis modules described below are implemented as workflow-accessible production components integrated into both the graphical interface and the MCP-based AI-agent framework. These capabilities include radial distribution functions $g(r)$, hydrogen-bond analysis, density profiling, RMSD/RMSF analysis, cavitation free-energy calculations, blue Moon constrained AIMD, slow-growth thermodynamic integration, and constant-potential electrochemical simulations. The representative interfacial case study shown in Fig. 5 and discussed in ref. 89 was produced using this integrated analysis pipeline.

4.2.1 Trajectory visualization and standard structural analysis. The trajectory environment supports real-time playback of molecular dynamics and geometry-optimization trajectories together with synchronized structural and energetic analysis. For geometry optimizations, convergence traces such as total energy and maximum force are displayed alongside the evolving

structure. For molecular dynamics trajectories, instantaneous temperature, kinetic energy, potential energy, and user-selected geometric descriptors can be monitored interactively.

Standard trajectory analyses include radial distribution functions (RDFs), mean squared displacement (MSD), velocity autocorrelation functions (VACF), density profiles, RMSD/RMSF analysis, coordination-number distributions, interatomic distance tracking, and structural clustering. The RDF implementation supports element-resolved pair correlations and running coordination-number integration,

$$n(r) = 4\pi\rho \int_0^r g(r') r'^2 dr', \quad (3)$$

while diffusion coefficients are extracted from the Einstein relation

$$\text{MSD}(\tau) = 2dD\tau. \quad (4)$$

For interfacial systems, CatGo further provides one-dimensional and two-dimensional density analysis for characterizing electric double layer (EDL) structure, water layering, and adsorbate distributions.^{90,91} Hydrogen-bond analysis includes geometric hydrogen-bond detection,^{92,93} hydrogen-bond autocorrelation functions, and extraction of hydrogen-bond lifetimes τ_{HB} , enabling quantitative characterization of interfacial water-network rigidity and transport behavior.⁹⁴

Additional analyses include orientational order parameters for interfacial water,

$$\langle \cos \phi \rangle(z), \quad (5)$$

where ϕ denotes the angle between the molecular dipole vector and the surface normal, as well as structural clustering and dimensionality-reduction techniques such as PCA,⁹⁵ t-SNE,⁹⁶ and UMAP.⁹⁷ These analyses enable identification of conformational basins, transition pathways, and rare-event dynamics in AIMD simulations.

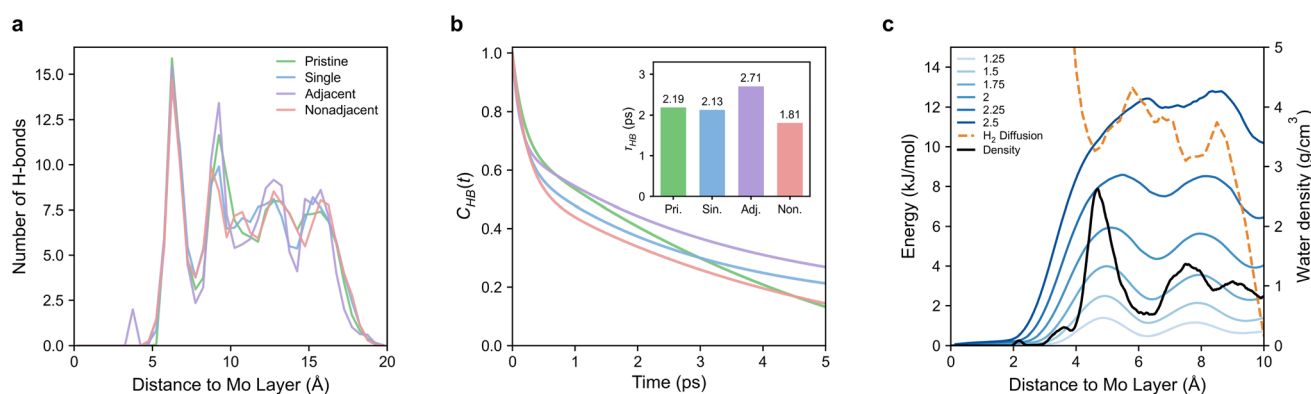


Fig. 5 Integrated trajectory-to-free-energy analysis suite for defective MoS₂-water interfaces, shown here as an application example from our related work in ref. 89. (a) Spatially resolved hydrogen bond count profiles along the surface normal for pristine, single-vacancy, adjacent-vacancy, and non-adjacent-vacancy configurations, showing enhanced hydrogen bond density near the inner and outer Helmholtz planes for the adjacent-vacancy surface. (b) Hydrogen bond autocorrelation functions $C(t)$ for interfacial water, with the inset summarizing extracted lifetimes, revealing a markedly longer-lived network for the adjacent-vacancy case. (c) Overlay of cavitation energy profiles $\Delta G_{\text{cav}}(R, z)$, interfacial water density $\rho(z)$, and the explicit H₂ diffusion free energy $\Delta F(\xi)$ obtained from Blue Moon constrained AIMD; the spatial alignment of barrier maxima with water-density peaks validates cavitation energy as a computationally efficient proxy for interfacial mass transport prediction.

4.2.2 Cavitation free energy analysis. The free energy cost associated with forming a solute-sized cavity within a solvent environment, commonly referred to as the *cavitation free energy*, is a central quantity in the Lum–Chandler–Weeks (LCW) theory of hydrophobic hydration.^{98–100} Beyond its fundamental role in liquid-state theory, cavitation thermodynamics has recently emerged as a physically meaningful descriptor for understanding hydrophobic solute transport across electrochemical interfaces, where solvent reorganization and interfacial water structuring strongly influence molecular accessibility to catalytic active sites.^{101–104} CatGo implements a spatially resolved cavitation free-energy framework directly from AIMD trajectories, enabling quantitative analysis of how surface morphology, defects, vacancies, and adsorbates reshape interfacial solvation thermodynamics.

For a spherical probe of radius R centered at position r , the cavitation probability $P_0(R, r)$ is defined as the probability that the probe volume contains no solvent molecules, typically water oxygen atoms. The corresponding cavitation free energy follows from the Boltzmann relation:

$$\Delta G_{\text{cav}}(R, z) = -k_{\text{B}}T \ln P_0(R, z), \quad (6)$$

where z denotes the distance from the surface along the slab normal. Physically, $\Delta G_{\text{cav}}(R, z)$ measures the local thermodynamic penalty required to reorganize the solvent environment sufficiently to accommodate a hydrophobic solute of size R . Larger values therefore indicate regions in which solvent structuring is more resistant to cavity formation and, consequently, less favorable for hydrophobic transport.

CatGo evaluates $P_0(R, z)$ by discretizing the simulation cell along the surface normal and systematically inserting spherical probe volumes throughout the interfacial region across all AIMD frames. For each spatial bin, the fraction of probe placements that remain free of solvent atoms is accumulated to obtain the local cavitation probability. The framework supports user-defined probe radii, spatial resolution, and solvent species selection, while periodic-boundary-condition corrections ensure proper treatment of probe overlaps near cell boundaries.

The resulting cavitation free-energy profiles exhibit strongly oscillatory behavior that directly reflects the layered structure of interfacial water near solid surfaces. In particular, maxima in $\Delta G_{\text{cav}}(R, z)$ occur near the inner and outer Helmholtz planes (IHP/OHP), where water density and hydrogen-bond ordering are highest, whereas minima emerge within density-depleted interfacial regions.^{98,101} As the probe radius increases, the cavitation penalty also increases, reflecting the increasing solvent reorganization cost associated with larger excluded volumes. According to LCW theory, small hydrophobic solutes exhibit an approximately linear scaling between cavitation free energy and cavity volume,

$$V = \frac{4}{3}\pi R^3, \quad (7)$$

which CatGo verifies through regression analysis of ΔG_{cav} versus probe volume across different interfacial environments.^{98,99}

To further connect cavitation thermodynamics with interfacial transport, CatGo evaluates the transport-related descriptor

$$\Delta G_{\text{cav}}(R) = \Delta G_{\text{cav}}^{\text{IHP}}(R) - \Delta G_{\text{cav}}^{\text{Stern}}(R), \quad (8)$$

which approximates the solvation contribution to the free-energy barrier associated with migration from the Stern layer into the inner Helmholtz plane. Because explicit constrained-AIMD free-energy calculations are computationally demanding, this cavitation descriptor provides a substantially more efficient proxy for estimating hydrophobic transport barriers at electrochemical interfaces.¹⁰¹

Importantly, CatGo integrates cavitation analysis directly with density profiling, hydrogen-bond analysis, and water-orientation characterization, enabling multiple interfacial descriptors to be visualized simultaneously within a common spatial coordinate system. This integration allows users to directly correlate solvent density fluctuations, hydrogen-bond rigidity, and cavity-formation thermodynamics with interfacial transport behavior and catalytic accessibility.

Fig. 5 summarizes this integrated analysis framework using a recent study of H_2 transport across a defective MoS_2 –water interface.⁸⁹ In the right panel, two independent CatGo analysis pipelines are combined into a single interfacial free-energy representation: (i) the cavitation free-energy profile $\Delta G_{\text{cav}}(R, z)$ obtained from equilibrium AIMD trajectories *via* eqn (6), and (ii) the explicit H_2 diffusion free-energy profile $\Delta F(\xi)$ computed through Blue Moon constrained AIMD. Both quantities are overlaid with the interfacial water-density profile $\rho(z)$ along the same spatial coordinate. The barrier maxima of $\Delta F(\xi)$ align closely with both the cavitation-energy peaks and the density maxima at the IHP and OHP, providing direct evidence that hydrophobic transport across electrochemical interfaces is strongly governed by solvent cavity-formation thermodynamics. This agreement further validates ΔG_{cav} as a computationally efficient descriptor for predicting interfacial transport barriers without requiring expensive constrained free-energy calculations for every solute species.

4.2.3 Hydrogen bond analysis. Hydrogen-bond dynamics play a central role in determining interfacial solvation structure, proton transport, and solvent-mediated electrochemical kinetics.¹⁰⁵ CatGo provides a multi-level hydrogen-bond analysis framework that combines geometric detection, dynamical lifetime analysis, and spatially resolved hydrogen-bond density characterization.¹⁰⁶

Hydrogen bonds are identified frame-by-frame using either the Baker–Hubbard or Wernet–Nilsson geometric criteria,^{92,93} with user-configurable distance and angular cutoffs under full periodic-boundary-condition treatment. The framework returns per-frame hydrogen-bond populations together with explicit donor–hydrogen–acceptor triplets, enabling direct construction of spatially resolved hydrogen-bond profiles throughout the electric double layer.

To characterize hydrogen-bond dynamics, CatGo evaluates the hydrogen-bond autocorrelation function

$$C(t) = \frac{\langle h(0)h(t) \rangle}{\langle h(0)^2 \rangle}, \quad (9)$$

where $h(t) = 1$ if a hydrogen bond exists at time t and 0 otherwise. The corresponding hydrogen-bond lifetime,

$$\tau_{\text{HB}} = \int_0^\infty C(t) dt, \quad (10)$$

provides a quantitative measure of solvent-network rigidity and interfacial water mobility. Longer lifetimes indicate more rigid and persistent hydrogen-bond networks, whereas shorter lifetimes correspond to dynamically labile water structures.

CatGo additionally computes spatial hydrogen-bond density distributions by assigning hydrogen bonds to spatial regions according to the midpoint of the donor-acceptor pair. This enables direct visualization of hydrogen-bond network organization across interfacial regions such as the Stern layer, inner Helmholtz plane, and outer Helmholtz plane.

As a representative application, Zhang *et al.*⁸⁹ employed this framework to investigate how sulfur-vacancy configurations on MoS₂ surfaces modify interfacial water structure and hydrophobic transport across the electric double layer. Fig. 5a shows the spatially resolved hydrogen-bond count profiles along the surface normal for pristine, single-vacancy, adjacent-vacancy, and non-adjacent-vacancy surfaces. Adjacent sulfur vacancies stabilize a rigid “O-down” chemisorbed water configuration, producing enhanced hydrogen-bond density within both the inner and outer Helmholtz planes while suppressing interlayer solvent exchange. Fig. 5b presents the corresponding hydrogen-bond autocorrelation functions $C(t)$ for interfacial water. The adjacent-vacancy interface exhibits a substantially slower decay than the pristine, single-vacancy, and non-adjacent-vacancy systems, while the inset lifetime analysis quantitatively confirms the same ordering, indicating the formation of a more rigid and long-lived interfacial hydrogen-bond network.

Fig. 5c further correlates these structural signatures with interfacial transport thermodynamics by overlaying the cavitation free-energy profiles $\Delta G_{\text{cav}}(R, z)$ computed for multiple probe radii, the interfacial water-density profile $\rho(z)$, and the explicit H₂ diffusion free-energy profile $\Delta F(\xi)$ obtained from Blue Moon constrained AIMD. The cavitation free-energy curves exhibit pronounced oscillatory features spatially aligned with the layered water structure, with maxima near the inner and outer Helmholtz planes where water density is highest. As the probe radius increases, the cavitation penalty increases systematically, reflecting the greater solvent-reorganization cost associated with larger excluded volumes. Most importantly, the explicit H₂ diffusion barrier follows the same spatial modulation as both the water-density and cavitation-energy profiles, demonstrating that solvent cavity formation governs hydrophobic transport across the interface. Together, Fig. 5a–c establish a direct connection between surface-defect topology, hydrogen-bond rigidity, interfacial solvation structure, and molecular transport behavior at electrochemical interfaces.

5 Automated workflow system

The CatGo workflow system operationalizes the architectural principles described in Section 2 by combining a unified scientific data model, software-agnostic workflow abstractions, and AI-native workflow construction within a single execution environment. Rather than treating workflow management as a collection of isolated scripts and scheduler wrappers, CatGo represents computational research pipelines as persistent, executable scientific graphs that connect structure construction, simulation execution, analysis, and AI-agent interaction within a shared state model. This section describes the workflow graph abstraction, node architecture, execution engine, and AI-driven workflow orchestration framework.

5.1 Workflow graph construction and validation

CatGo workflows are represented as DAGs, where each node corresponds to a scientific task and each edge carries typed data between task ports. Typical workflow nodes include structure generation, geometry optimization, adsorption-site placement, molecular dynamics, electronic-structure analysis, free-energy evaluation, and machine-learning inference. Within the visual workflow editor, users construct workflows interactively through drag-to-connect operations, while the underlying graph engine continuously validates graph consistency and execution readiness in real time.

Validation is designed around scientific workflow integrity rather than low-level graph correctness alone. Blocking errors such as cyclic dependencies, disconnected execution paths, or unresolved mandatory inputs prevent workflow submission until corrected, whereas non-blocking inconsistencies, including optional port mismatches or incomplete analysis branches, are surfaced through warning indicators that allow iterative workflow development without interrupting exploratory construction. This approach enables rapid workflow prototyping while preserving execution reliability for production calculations.

Graph validation establishes structural integrity, but it does not adjudicate the scientific validity of the underlying physics. CatGo treats the choice of pseudopotential, k -point density, smearing, and convergence thresholds as a human-in-the-loop responsibility rather than a decision the platform makes automatically. Calculation-type presets and engine defaults, together with recommended PAW potentials assembled in the POSCAR species order, provide a physically reasonable starting point; however, before any job is dispatched, the complete generated input files (INCAR, KPOINTS, and POTCAR) are surfaced in an editable preview, and the job is submitted only after the user has inspected and confirmed them. CatGo therefore makes every scientific input explicit, inspectable, and editable rather than hiding it behind opaque automation: the platform assists with sensible defaults and full transparency, while the final judgment of scientific correctness remains with the researcher.

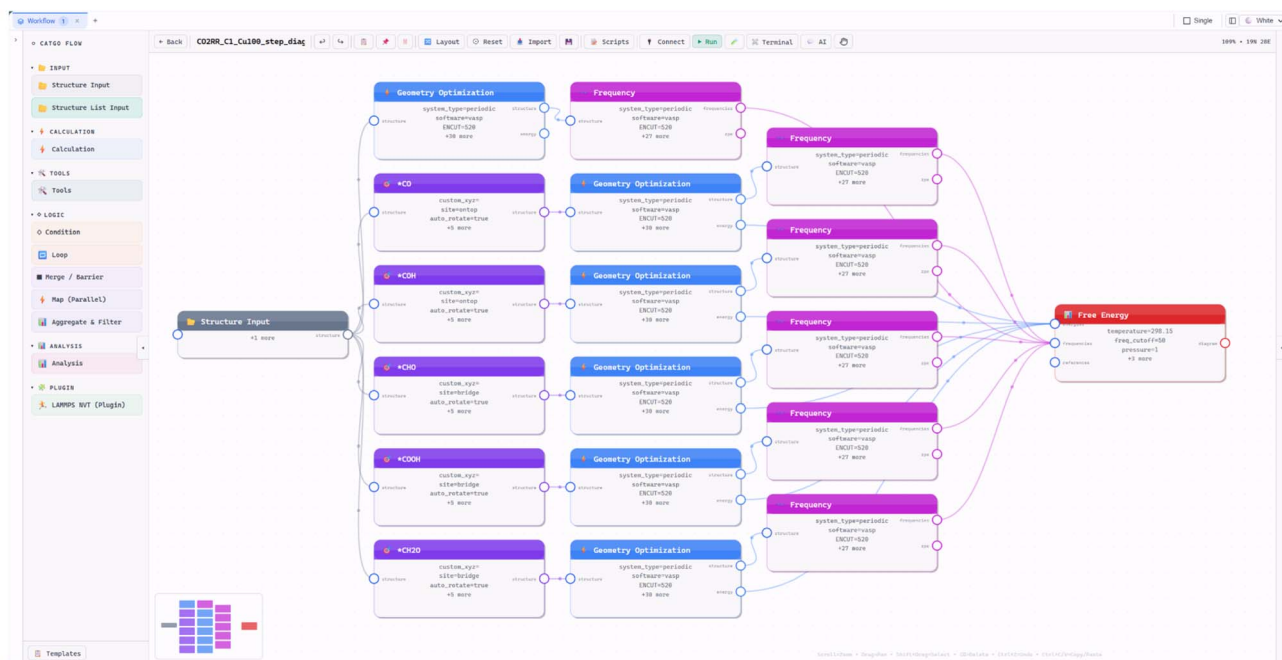
The graphical editor provides direct visual interaction with the workflow graph through an SVG-based canvas supporting

drag-to-connect ports, inline parameter editing, undo/redo operations, copy/paste with connection preservation, grid snapping, minimap navigation, and automatic workflow persistence. Workflows can additionally be exported as self-

contained JSON representations for sharing, version control, and reproducibility.

Fig. 6 illustrates the workflow construction environment at two complementary levels. Fig. 6a shows graph-level workflow

a



b

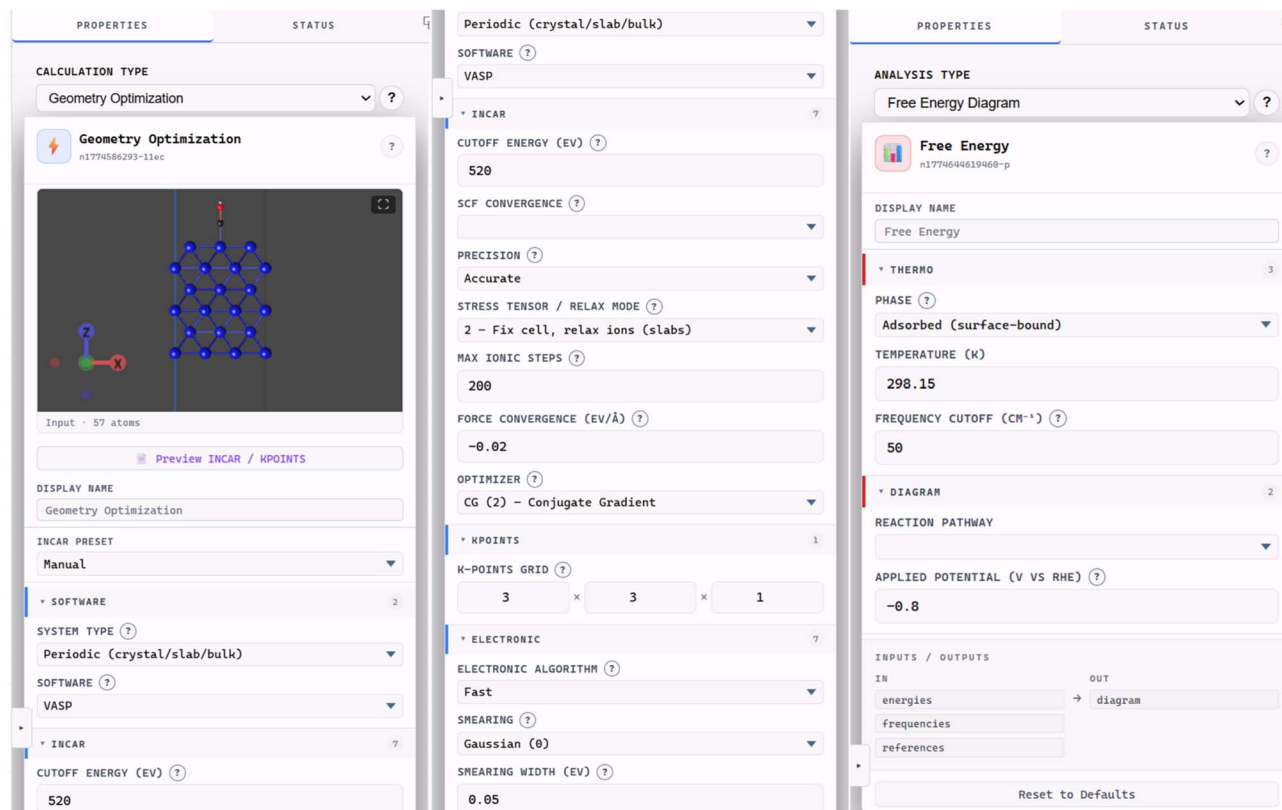


Fig. 6 Visual workflow editor in two enlarged panels. (a) Graph-level construction. (b) Node-level parameterization for execution-ready workflows.

composition, where scientific tasks are connected into executable computational pipelines through typed dataflow edges. Fig. 6b highlights node-level parameterization, demonstrating how calculation settings, execution parameters, and backend-specific options are configured within the same visual environment without requiring direct script editing.

5.2 Unified node architecture

A central architectural decision in CatGo is the adoption of a unified node abstraction in which workflow nodes correspond to scientific operations rather than specific software implementations. For example, a geometry-optimization node represents the abstract task of structural relaxation independent of whether the underlying backend is VASP, CP2K,⁴⁹ ORCA,⁵⁰ xTB,⁶⁴ or a machine-learning potential. Backend selection is instead specified through node-level execution parameters, allowing workflows to transition between computational engines without changing graph topology or downstream dependencies.

This abstraction separates scientific intent from computational implementation and enables flexible backend substitution during workflow development. A workflow initially prototyped using low-cost methods such as xTB or machine-learning potentials can therefore be upgraded to production DFT calculations by modifying backend parameters alone, while preserving the surrounding workflow logic, data contracts, and analysis structure.

At the interface level, node compatibility is enforced through typed input/output handles together with schema-validated parameter definitions. Consequently, workflows remain backend-agnostic while preserving strict consistency of data propagation and execution semantics. Table 2 summarizes the complete node library organized by functional category.

5.2.1 Workflow dispatch and execution routing. When a node becomes execution-ready, the CatGo dispatch engine classifies the task and routes it to the appropriate execution backend. Lightweight structure operations, logical transformations, and rapid analyses execute directly within the Python backend with low latency. More computationally demanding electronic-structure analyses are executed as asynchronous backend tasks, while DFT calculations generate backend-specific input files and are dispatched automatically to HPC systems through SSH/SFTP communication and scheduler integration.

DFT workflows currently support VASP,^{46,47} CP2K,^{48,49} ORCA,⁵⁰ and LAMMPS,⁵¹ together with declarative task-node interfaces for Gaussian,⁶³ Quantum ESPRESSO,⁵³ and GRO-MACS.⁸⁷ Machine-learning potential workflows including MACE, CHGNet, and M3GNet can execute either locally on GPU-enabled workstations or through GPU-equipped HPC resources.

5.2.2 HPC job lifecycle and provenance. For HPC-bound calculations, CatGo manages the complete computational lifecycle from input generation to result retrieval. Node parameters and upstream structural data are automatically translated into backend-native input formats, including INCAR/POSCAR/KPOINTS/POTCAR generation for VASP workflows. Prior to

execution, users may inspect and modify generated scheduler scripts through an integrated Monaco-based editor supporting SLURM and PBS environments.

After workflow submission, CatGo transfers input files to the target HPC system through SFTP and submits calculations *via* scheduler interfaces such as sbatch or qsub. The execution engine subsequently monitors scheduler state through periodic polling and synchronizes workflow status with the frontend in real time. Upon completion, output files are downloaded automatically, parsed into structured workflow objects, and propagated downstream for analysis and subsequent workflow execution.

To improve reliability in production HPC environments, CatGo integrates provenance tracking directly into the execution engine. Each workflow node records immutable execution metadata including backend type, scheduler job identifiers, working directories, timestamps, parser state, and execution parameters. Recoverable failures such as transient SSH disconnects or scheduler-query interruptions trigger automatic state resynchronization, whereas non-recoverable failures preserve logs and execution artifacts for debugging and selective rerun of affected workflow branches. This provenance-aware design substantially improves reproducibility and reduces wasted computational allocation during large screening campaigns.

5.2.3 High-throughput workflow expansion. High-throughput catalyst discovery workflows often require evaluating large numbers of candidate structures through identical computational pipelines. CatGo supports this workflow pattern through native fan-out/fan-in graph expansion using map and aggregate workflow primitives. Batch structure-generation and adsorption-site expansion nodes automatically enumerate combinations of structures, adsorbates, and adsorption sites according to

$$N_{\text{out}} = N_{\text{structures}} \times N_{\text{adsorbates}} \times N_{\text{sites}}, \quad (11)$$

after which downstream computational branches are replicated automatically and executed concurrently.

To prevent excessive scheduler load during large campaigns, CatGo incorporates user-configurable concurrency limits through a `max_parallel` semaphore controlling simultaneous HPC submissions. Aggregation nodes subsequently collect results across all branches, rank candidates according to user-defined descriptors such as adsorption energy, overpotential, or band gap, and optionally propagate only top-ranked structures into subsequent workflow stages.

5.2.4 File-first campaign orchestration. Beyond per-workflow graph expansion, CatGo provides a complementary *campaign* mode for multi-stage studies whose state is held in a human-readable folder and Markdown tree rather than in an opaque, database-backed store. A campaign is laid out as a directory hierarchy in which every stage is its own folder and every calculation is a leaf with a recognizable, non-hashed name, so that the entire study remains navigable and resumable directly from disk. Each level pairs a Markdown plan with companion index and status files: a top-level plan defines the scientific objective and the ordered stages, per-stage README

and INDEX files record what each stage contains and the reasoning behind it, and a running log captures manual interventions and lessons learned. Because the campaign state is

plain text, the researcher and an AI agent operate on the same files, and a study can be resumed after an interruption (for example, after a session restart) simply by re-reading the tree.

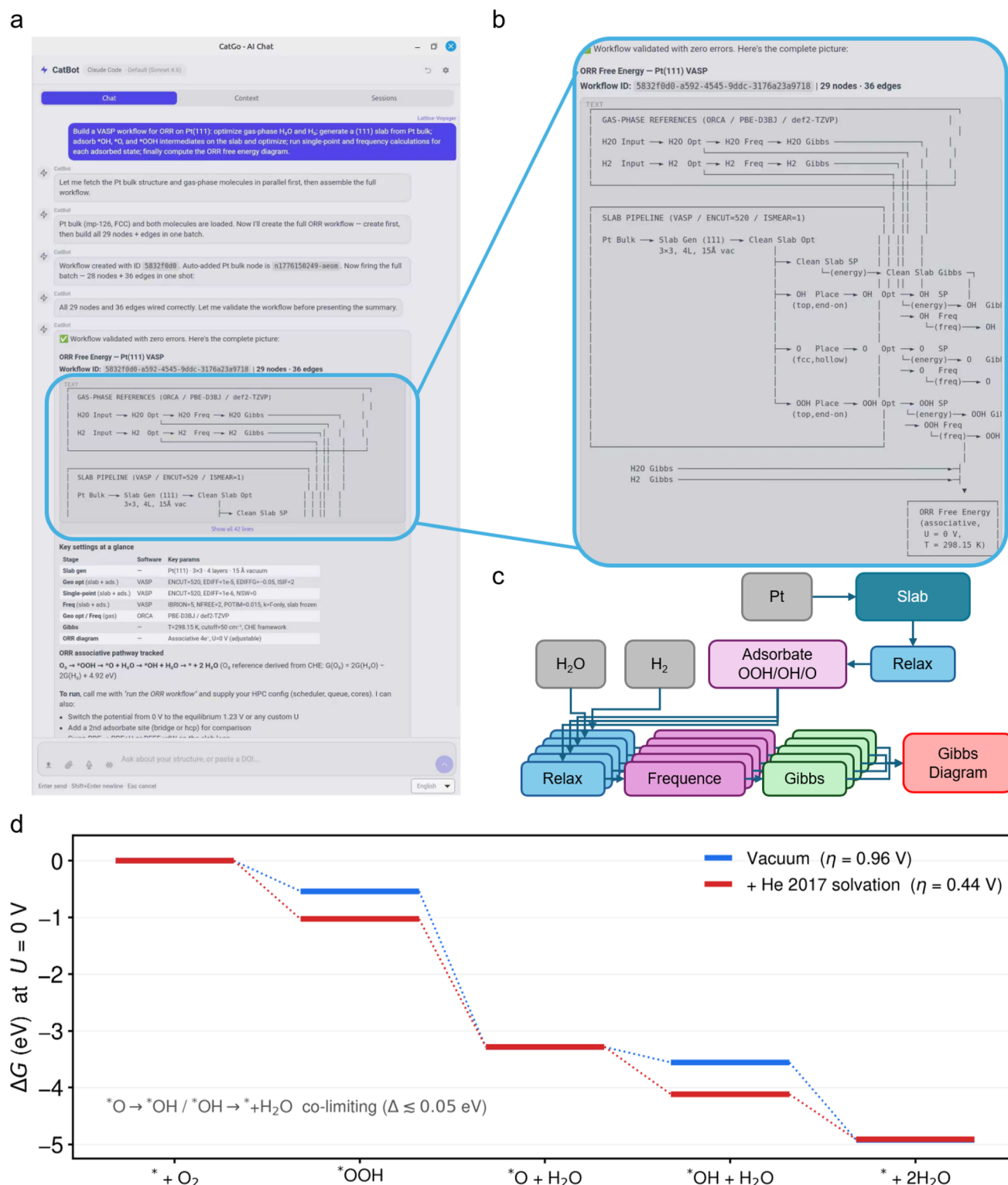


Fig. 7 AI-driven workflow construction. (a and b) Chat-driven workflow generation in the CatGo interface, where a natural-language request and the corresponding implemented workflow appear in the same view. (c) Simplified auto-generated DAG showing the resulting branch structure. (d) The resulting free-energy output illustrating CatGo's ability to connect graph construction, calculation execution, and thermodynamic analysis within the same visual environment.

A typical campaign proceeds through four stages: candidate generation, funnel-style screening that carries only the surviving structures forward, analysis of the retained candidates, and automated report assembly. Each stage consumes the outputs of the previous one and writes its own, so the provenance of every decision is explicit on disk rather than hidden in scheduler or database state. This file-first mode is suited to multi-stage, high-throughput screening studies and to long-running studies that an agent monitors and revises over time, with the plain-text tree keeping the whole study navigable and resumable directly from disk.

5.3 Catalysis reaction analysis templates

CatGo includes built-in workflow templates for common electrocatalytic reactions based on the computational hydrogen electrode (CHE) formalism.¹⁰⁷ The `free_energy` analysis node computes Gibbs free-energy corrections according to

$$\Delta G = \Delta E_{\text{DFT}} + \Delta \text{ZPE} - T\Delta S, \quad (12)$$

and derives corresponding reaction energetics and overpotentials.

Pre-configured workflow templates currently include oxygen evolution reaction (OER),^{108,109} hydrogen evolution reaction (HER),¹¹⁰ CO₂ reduction,¹¹¹ and nitrogen reduction reaction (NRR)¹¹² pipelines. These templates provide automated free-energy-diagram construction, volcano-plot generation, and scaling-relation analysis.¹¹³

In addition to native CatGo workflows, the platform supports importing workflow templates from established computational chemistry ecosystems including atomate2 and quacc, enabling interoperability with existing workflow infrastructures while preserving CatGo's visual and AI-driven orchestration capabilities.

5.4 Project management and AI-driven workflow generation

Projects in CatGo function as organizational containers that aggregate workflows, structures, execution states, and analysis results into unified research workspaces. The project dashboard provides workflow-state visualization, progress tracking, structure statistics, energy rankings, convergence summaries, and graphical analysis outputs including free-energy diagrams and volcano plots. This organization model is particularly important for large-scale catalyst-screening campaigns involving many concurrent workflow branches.

A defining feature of CatGo is the integration of AI-agent-driven workflow construction directly into the workflow environment. Through MCP-enabled interfaces, coding-capable agents such as Claude Code,⁵⁴ Codex CLI,⁵⁵ and Gemini CLI⁵⁶ can construct, modify, validate, and execute workflows through natural-language interaction. Rather than generating standalone scripts, the AI agent operates directly on the persistent workflow graph and shares the same synchronized state as the graphical interface.

Fig. 7 demonstrates this workflow paradigm. Fig. 7a and b show chat-driven workflow construction within the CatGo

interface, where natural-language instructions and the resulting executable workflow coexist within the same interactive environment. Fig. 7c presents the automatically generated DAG structure produced by the AI agent, while Fig. 7d shows the resulting free-energy analysis output. Together, these examples illustrate how CatGo unifies AI-assisted workflow construction, heterogeneous computation, and thermodynamic analysis within a single reproducible scientific workflow framework.

6 AI-assisted interface

CatGo integrates an AI-assisted interaction layer that enables natural-language control over both the 3D structure environment and the computational workflow system. Unlike conventional graphical interfaces that require extensive manual parameterization and repetitive GUI operations, the CatGo interface is designed around an AI-native interaction model in which coding-capable large language model (LLM) agents can directly manipulate structures, construct workflows, monitor execution, and perform analysis through structured tool calls. The overall design objective is to reduce workflow-authoring overhead sufficiently that AI-driven construction of medium-scale computational DAGs (~10–30 nodes) becomes practical for realistic catalysis workflows. Quantitative benchmarks for token efficiency and workflow-authoring round-trip reduction are presented in Section 6.2.1.

6.1 Real-time workflow monitoring

CatGo provides real-time workflow monitoring through a persistent WebSocket synchronization layer connecting the backend execution engine with both the graphical editor and AI-agent interfaces. Workflow execution events are streamed continuously, allowing users and AI agents to observe node-level state transitions, execution progress, and calculation outputs within the same synchronized workflow environment.

Internally, workflow execution follows a fine-grained task-state lifecycle spanning local preparation, remote HPC execution, result collection, and terminal execution outcomes. These internal states are mapped onto a simplified user-facing taxonomy including *pending*, *queued*, *running*, *completed*, *failed*, *paused*, and *skipped*, allowing researchers to track workflow progress visually while preserving detailed backend execution resolution for retry logic, provenance analysis, and AI-agent reasoning.

Completed workflow nodes expose their results directly within the workflow editor. Depending on node type, these outputs include convergence traces, UV-vis spectra, density-of-states plots, VASP self-consistent-field convergence monitors, free-energy diagrams, surface-energy analyses, Wulff constructions,¹¹⁴ nudged elastic band (NEB) profiles,^{115,116} vibrational-mode inspectors, and thermodynamic corrections. Structural outputs can additionally be opened directly within the integrated 3D structure viewer for interactive analysis.

6.2 AI-driven workflow construction via MCP

A defining feature of CatGo is its ability to expose the entire workflow system through a unified AI-operable interface.

Because the structure editor, workflow engine, analysis framework, and execution backend share the same persistent workflow graph and scientific data model, an AI agent can construct, validate, execute, and monitor complete computational workflows without requiring manual intervention or external scripting.

CatGo implements a MCP server⁵⁷ that allows CLI-based coding agents, including Claude Code,¹¹⁷ Codex CLI, and Gemini CLI, to interact directly with the workflow environment. Workflow construction and execution operations are unified within the `catgo_workflow` tool interface, while complementary MCP tools expose structure editing, catalysis analysis, visualization control, DFT input generation, and geometry manipulation capabilities across multiple scientific modules.

Each workflow mutation operation returns a compact graph snapshot summarizing workflow topology, node connectivity, and key parameters. Unlike raw JSON graph representations, these snapshots are topologically ordered and structured specifically for efficient LLM reasoning, enabling agents to maintain workflow awareness with substantially reduced context cost.

CatGo supports two complementary AI interaction channels that operate on the same server-authoritative workflow graph. The first is an in-browser conversational interface integrated directly into the graphical editor. In this mode, workflow-generation requests, structure-editing operations, and AI-generated outputs appear within the same interface as the visual workflow graph and structure viewer. The second interaction mode exposes the MCP tool layer directly to external CLI-based coding agents. Importantly, both interaction channels ultimately invoke the same backend workflow engine and validation pipeline, ensuring that workflow state, provenance tracking, and execution semantics remain fully synchronized regardless of the interaction modality.

This unified architecture is particularly advantageous for high-throughput catalysis workflows. An AI agent can automatically generate adsorption-screening workflows, enumerate adsorption sites, construct geometry-optimization branches, connect free-energy-analysis nodes, and execute the resulting DAG programmatically without manual graph editing. The design therefore extends recent trends in tool-augmented chemistry agents⁷ toward fully executable computational chemistry workflow systems.

6.2.1 Authoring efficiency: token cost and workflow round-trip reduction. To quantify the efficiency of the CatGo AI-authoring interface, we evaluate two deterministic properties

of the workflow interaction layer: (i) the token cost associated with exposing workflow state to the AI agent, and (ii) the number of tool-call round-trips required to construct representative catalysis workflows.

Benchmarks were performed using the Pt(111) ORR workflow described in Section 7, consisting of 24 workflow nodes spanning slab generation, adsorption branches (OH*, O*, OOH*), geometry optimization, vibrational analysis, gas-phase reference calculations, and terminal free-energy analysis. The benchmark script and corresponding graph dictionary are included in the CatGo repository.

6.2.1.1 Token cost of graph-state exposure (measured). We compared four serialization strategies for exposing the same 24-node DAG to an LLM agent using the `cl100k_base` tokenizer. Results are summarized in Table 5. The CatGo graph snapshot requires only 805 tokens, corresponding to a $1.62\times$ reduction relative to compact JSON serialization and a $2.80\times$ reduction relative to pretty-printed JSON. Although topology-only graph representations achieve even lower token counts, they omit the parameter-level information necessary for workflow validation and scientific reasoning. The primary advantage of the CatGo snapshot format is therefore not only token compression, but also preservation of execution order, parameter visibility, and workflow semantics in a representation optimized for AI-agent reasoning.

6.2.1.2 Round-trip reduction from batch workflow actions (measured). Constructing the same 24-node ORR workflow without workflow batching requires one workflow-creation operation together with individual node-addition and edge-connection calls, resulting in on the order of fifty separate MCP interactions. By contrast, the CatGo batch action allows all graph mutations to be emitted within a single MCP request, reducing workflow construction to only two interactions: one `batch` call followed by one `validate_workflow` call. This corresponds to an order-of-magnitude reduction in workflow-authoring round-trips and substantially improves the practicality of AI-driven graph construction for medium-scale catalysis workflows.

6.2.1.3 Bidirectional synchronization. Workflow mutations issued by AI agents are synchronized to the graphical editor through the same WebSocket infrastructure used for workflow execution monitoring. Because both the GUI and AI agents operate on the same server-authoritative workflow graph, all graph mutations pass through the same validation and persistence pipeline. Consequently, workflow edits remain fully synchronized across AI and graphical interfaces without requiring additional reconciliation logic. Quantitative latency

Table 5 Measured token cost of exposing the 24-node Pt(111) ORR DAG (Section 7) to an LLM agent under four serialization strategies. Tokens are counted with the `cl100k_base` BPE tokenizer; the CatGo snapshot is the actual output of `_graph_snapshot` invoked on the exported graph dictionary. See `measure_authoring_efficiency.py` for the reproduction script

Serialization strategy	Chars	Tokens	× vs. compact JSON
Full JSON dump (pretty/indented)	6490	2250	0.58×
Full JSON dump (compact, single-line)	3762	1308	1.00× (ref.)
CatGo <code>_graph_snapshot</code>	2062	805	1.62× smaller
Topology-only (nodes + edges, no params)	1457	494	2.65× smaller

measurements for distributed deployments are deferred to future systems-oriented studies.

Together, these benchmarks demonstrate that CatGo reduces both the context overhead and interaction complexity associated with AI-driven workflow construction, enabling practical autonomous authoring of medium-scale computational chemistry workflows.

6.3 Natural-language workflow construction and context-aware assistance

Beyond direct workflow editing, CatGo enables conversational workflow generation through natural-language instructions. The AI assistant is connected directly to structure-control tools, workflow APIs, and execution-aware context retrieval systems, allowing users to construct and manipulate computational workflows through high-level scientific descriptions rather than low-level scripting or manual GUI interaction.

For example, a request such as “Build a workflow to compute the ORR free-energy diagram on Pt(111) using VASP” automatically generates the corresponding workflow graph, initializes relevant node parameters, identifies missing inputs, and configures free-energy-analysis branches through the MCP workflow interface. Similarly, structure-oriented instructions such as “Generate a 2×2 Pt(111) slab, adsorb OH* on the bridge site, hide labels, and export a white-background image” are decomposed into sequential structure and visualization tool calls executed automatically by the AI assistant.

CatGo additionally incorporates retrieval-augmented generation (RAG)¹¹⁸ together with live structure context including chemical composition, lattice parameters, atom selections, and workflow state. This enables context-aware assistance such as recommending *k*-point meshes based on supercell dimensions, identifying unusual bond lengths, or suggesting appropriate computational settings based on the active workflow environment.

6.4 Multimodal interaction (experimental)

In addition to text-based interaction, CatGo experimentally supports multimodal interaction through webcam-based gesture recognition and Whisper-based¹¹⁹ voice commands. These features enable hands-free structure rotation, navigation, and basic workflow interaction, and are currently under active development as exploratory extensions of the AI-assisted interface framework.

7 Application: automated Pt(111) ORR free energy workflow

To demonstrate the end-to-end capabilities of CatGo, we constructed an automated oxygen reduction reaction (ORR) workflow on the Pt(111) surface. Pt(111) is the canonical benchmark system for ORR electrocatalysis because it represents the active facet of Pt-based fuel-cell cathodes and serves as the reference system on which the CHE framework was originally established.¹⁰⁷ Its adsorption energetics, scaling relations, and overpotential behavior are extensively documented in the literature,

making it an ideal reference case for evaluating workflow-generation and thermodynamic-analysis capabilities.

The purpose of the present demonstration is not to report a manually assembled benchmark calculation, but rather to show that CatGo can automatically generate, validate, execute, and serialize an integrated computational catalysis workflow spanning bulk import, slab construction, adsorbate placement, geometry optimization, vibrational analysis, thermochemical correction, and ORR free-energy assembly within a single executable DAG. This example therefore exercises all major CatGo subsystems simultaneously, including the visual workflow editor, unified node architecture, HPC execution engine, thermodynamic-analysis framework, and AI-compatible workflow representation.

7.1 Workflow overview

The complete ORR workflow was constructed directly within CatGo's visual workflow editor. The workflow graph consists of one bulk Pt input branch, one slab-construction stage, three adsorbate branches corresponding to OOH*, O*, and OH* intermediates, several gas-phase reference branches, and a terminal Free Energy node that automatically assembles the ORR free-energy diagram from upstream calculation outputs.

Each adsorbate branch follows the same thermodynamic workflow structure:

geometry optimization → frequency calculation
→ Gibbs correction

Gas-phase reference molecules are treated through independent but structurally analogous branches, and all electronic energies and vibrational contributions subsequently converge into the terminal thermodynamic-analysis node at $T = 298.15$ K.

Importantly, the workflow graph itself encodes the complete scientific logic of the ORR calculation. Rather than relying on external scripts or manually assembled spreadsheets, CatGo preserves the full dependency chain connecting slab generation, adsorbate placement, DFT calculations, vibrational corrections, and free-energy analysis within the same executable graph representation. The full workflow graph is provided in the SI (Fig. S3), while Fig. 7d shows the resulting free-energy diagram generated directly from the workflow state.

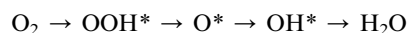
7.2 Bulk structure and slab construction

The workflow begins with a `structure_input` node containing the FCC Pt bulk structure. A dedicated Slab Gen node subsequently constructs the Pt(111) surface used throughout the ORR workflow. In the present configuration, the slab is generated with Miller index (111), eight atomic layers, and 15 Å of vacuum spacing. The generated slab is passed directly to downstream adsorption branches through CatGo's unified workflow data model without intermediate file conversion or external preprocessing.

This design illustrates a central architectural principle of CatGo: structure construction is treated as an explicit workflow operation rather than a separate preprocessing stage. As a result, slab-generation parameters remain embedded within the workflow graph itself and can be inspected, modified, versioned, and reused within the same reproducible environment.

7.3 Adsorbate placement

Three ORR intermediates, OOH*, O*, and OH*, were introduced using CatGo's Adsorbate nodes. Each adsorbate was selected from the built-in electrocatalysis preset library and positioned at atop adsorption sites on the Pt(111) surface. The resulting workflow corresponds to the associative ORR pathway:



Because adsorbate placement is represented explicitly as a graph node, the complete sequence

```
slab generation → adsorbate placement
                  → geometry optimization
                  → thermochemical analysis
```

remains visible and reproducible for every branch of the ORR pathway. This explicit representation becomes particularly important for catalyst-screening workflows involving many adsorbates, adsorption sites, and surface compositions.

7.4 DFT workflow configuration

All geometry optimizations and vibrational calculations in the present workflow are configured for VASP execution. The workflow uses a shared DFT parameter set centered on VASP with ENCUT = 450 eV, while vibrational calculations employ finite-difference phonon analysis (IBRION = 5) to generate the frequencies required for Gibbs free-energy corrections. The full set of converged computational parameters used in the Pt(111) ORR demonstration is summarized in Table 6.

The slab is built with a user-specified number of atomic layers (here four), given either as an explicit count or as a thickness in Å; the builder reports the layers it detects for interactive control but does not auto-select the count. For atom constraints, CatGo supplies only a basic initial setting rather than a resolution of the underlying scientific choice. By default a chosen number of bottom layers is fixed to represent the bulk while the upper layers and the adsorbate are relaxed (here the bottom two of the four layers); for the frequency stage the default relaxes only the adsorbate, so that just its vibrational modes enter the partial Hessian. There is no automatic reaction-aware freezing: an adsorbate is free only because it lies above the fixed region, not through active-site detection. Consistent with CatGo's human-in-the-loop handling of scientific inputs, these constraints are a starting point that the user is expected to inspect and adjust with their own judgment, not a decision the platform makes on the user's behalf.

A key feature of the workflow representation is that the computational backend and execution parameters are embedded directly within the workflow graph itself. Consequently, the DAG stores not only the scientific dependency graph, but also the execution-level provenance required for computational reproducibility.

Gas-phase reference calculations are likewise retained inside the same workflow environment rather than being delegated to external scripts or manually managed calculations. This ensures that all thermodynamic contributions entering the final ORR analysis remain explicitly linked through validated workflow dependencies, substantially reducing bookkeeping errors and improving provenance traceability.

7.5 Automated free-energy analysis

The terminal Free Energy node automatically collects electronic energies and vibrational data from upstream branches and evaluates the ORR free-energy diagram within the CHE framework.¹⁰⁷

Under the CHE formalism, the ORR pathway $^* + \text{O}_2 \rightarrow \text{OOH}^* \rightarrow \text{O}^* \rightarrow \text{OH}^* \rightarrow ^* + \text{H}_2\text{O}$ is evaluated at the equilibrium

Table 6 Computational parameters for the converged Pt(111) ORR workflow

Parameter	Value
Exchange–correlation functional	RPBE (GGA = RP)
Plane-wave cutoff (ENCUT)	450 eV
PAW potentials	PAW–PBE: Pt (5d ⁹ 6 s ¹), O (2s ² 2p ⁴)
k-point mesh	Γ-centered 4 × 4 × 1
Smearing	Methfessel–Paxton (ISMEAR = 1), σ = 0.2 eV
Electronic convergence (EDIFF)	1 × 10 ^{−5} eV
Force convergence (EDIFFG)	−0.03 eV Å
Spin polarization (ISPIN)	2 (collinear)
Slab model	Pt(111) 3 × 3, four atomic layers
Constraints	Bottom two layers fixed; top two layers + adsorbate relaxed
Vacuum spacing	≈ 29 Å
Dipole correction	Not applied (vacuum > 25 Å)
Adsorption sites	O* fcc hollow; OH*, OOH* atop
Thermodynamic framework	CHE, U = 1.23 V vs. RHE; references H ₂ (1 bar), H ₂ O (liquid, p = 0.035 bar); O ₂ via O ₂ + 2H ₂ → 2H ₂ O
Vibrational analysis	Finite difference (IBRION = 5), adsorbate-only Hessian
Solvation	Literature explicit-water corrections (He <i>et al.</i> ¹²⁰): OH* −0.57 eV, OOH* −0.49 eV

potential $U = 1.23$ V versus the reversible hydrogen electrode (RHE). Within this framework, the potential-determining step defines the theoretical overpotential η .

For the present Pt(111) demonstration, the workflow assembles the ORR free-energy diagram on a Pt(111)(3 × 3) four-layer slab. At the bare-DFT (vacuum) level, the terminal node reports adsorption free energies (referenced to liquid water) $\Delta G_{\text{O}} = 1.64$ eV, $\Delta G_{\text{OH}} = 1.36$ eV, and $\Delta G_{\text{OOH}} = 4.38$ eV, a potential-determining step $\text{*O} \rightarrow \text{*OH}$, and a theoretical overpotential $\eta = 0.96$ V. The corresponding bare binding energies ($\Delta E_{\text{O}} = 1.62$, $\Delta E_{\text{OH}} = 1.12$ eV) reproduce the canonical Pt(111) values of Nørskov *et al.*¹⁰⁷ (1.57 and 1.05 eV), confirming that the adsorption sites and energetics are correctly described. The residual difference from the experimental overpotential (≈ 0.45 V) originates from adsorbate–water solvation, which is absent at the bare-DFT level. Applying the explicit-water solvation corrections reported for Pt(111) by He *et al.*¹²⁰ (−0.57 eV for *OH and −0.49 eV for *OOH) to these free energies lowers the theoretical overpotential to $\eta = 0.44$ V, in quantitative agreement with both experiment (≈ 0.45 V) and the canonical CHE description of Pt(111).¹⁰⁷ At this solvation level the $\text{*O} \rightarrow \text{*OH}$ and $\text{*OH} \rightarrow \text{*} + \text{H}_2\text{O}$ steps become co-limiting (within ≈ 0.05 eV), consistent with Nørskov *et al.*, whose two terminal proton transfers likewise differ by only ≈ 0.02 eV (0.45 and 0.43 eV at $U = 1.23$ V);¹⁰⁷ the predicted $\eta = 0.44$ V is therefore robust to the precise value of the *OH solvation correction.

Beyond reproducing the established overpotential and potential-determining step, the key methodological point is that this free-energy analysis, both at the bare-DFT level and after applying literature solvation corrections, is generated automatically from validated workflow outputs rather than from external post-processing scripts or manual spreadsheet assembly. Once the upstream calculations complete, CatGo assembles the ORR free-energy diagram directly from the workflow state through graph-validated dependencies connecting energies, vibrational corrections, and reference states.

7.6 Workflow reproducibility and serialization

The complete ORR workflow can be exported as a single JSON artifact containing the full DAG topology, node parameters, DFT settings, slab specifications, adsorbate configurations, thermodynamic conventions, and execution metadata. Importing this JSON file into a separate CatGo instance reconstructs the workflow automatically, enabling one-click workflow reproduction and sharing.

Compared with conventional computational catalysis pipelines built from loosely connected scripts and manually managed calculation directories, the CatGo serialization framework makes workflow topology, execution parameters, and scientific dependencies fully explicit and inspectable. The Pt(111) ORR workflow JSON is deposited on Zenodo (see Data Availability).

Additional serialization examples, node-library references, HPC portability demonstrations, and workflow import/export mechanisms are provided in the SI. In particular, Section S3 presents a complete 26-node MACE-based reproduction of the multi-faceted Ni catalyst study reported by Kreitz *et al.*,¹ while Section S6 demonstrates equivalent workflow serialization and execution for ORCA-based molecular reaction workflows and TD-DFT calculations.

7.7 Demonstration scope and literature context

The scope of the present Pt(111) demonstration is intentionally focused on workflow-construction fidelity rather than exhaustive numerical benchmarking. Specifically, the objective is to demonstrate that CatGo can generate, validate, serialize, and execute a complete ORR workflow spanning slab generation, adsorbate placement, geometry optimization, vibrational analysis, gas-phase references, and terminal CHE free-energy assembly within a single reproducible DAG.

The present manuscript reports the converged overpotential and its solvation dependence (above); exhaustive convergence testing and a detailed mechanistic analysis are left to future work.

To provide an executed literature benchmark using the same workflow-generation framework, Section S3.6 presents an end-to-end reproduction of the multi-facet Ni catalyst study of Kreitz *et al.*¹ using a 26-node CatGo workflow together with a dual-MLP MACE backend. The reproduced workflow recovers the reported surface energies, Wulff morphology, hydrogen adsorption energetics, coverage dependence, and $\text{CO}^* \rightarrow \text{C}^* + \text{O}^*$ dissociation barrier within reasonable agreement to the reference study while executing entirely on a single Apple-Silicon CPU without GPU, HPC, or commercial DFT software requirements.

To contextualize the Pt(111) workflow scientifically, Table 7 summarizes the canonical literature reference values associated with the CHE description of ORR on Pt(111). Pt(111) is known to lie near the top of the ORR volcano because its OH^* adsorption strength approaches the optimal binding regime while still obeying the characteristic OH^*/OOH^* scaling relation.^{107–109}

Table 7 Literature benchmark values for ORR on Pt(111) under the CHE framework. The CatGo workflow reproduces the reference overpotential once literature solvation corrections for the adsorbed intermediates are included (see text). Exhaustive convergence testing and detailed mechanistic analysis are left to future work

Descriptor	Literature value	Source
η_{ORR} on Pt(111)	$\approx 0.4\text{--}0.5$ V	107
$\Delta G_{\text{OOH}} - \Delta G_{\text{OH}}$ (scaling)	$\approx 3.2 \pm 0.2$ eV	108 and 109
Potential-determining step	$\text{*O} \rightarrow \text{*OH}$ ($\text{*OH} \rightarrow \text{H}_2\text{O}$ within 0.02 eV)	107
CHE reference correction	$U = 1.23$ V vs. RHE	107 and 108

The purpose of Table 7 is to establish the scientific target that the CatGo workflow reproduces once explicit-water solvation is included (the overpotential and the near-degenerate $^*O \rightarrow ^*OH/^*OH \rightarrow ^* + H_2O$ determining step), while exhaustive convergence testing is left to future work.

Several aspects of the present work are intentionally left outside the scope of exhaustive validation. First, the Pt(111) ORR workflow is presented primarily as a demonstration of workflow construction, serialization, and execution fidelity rather than as a production electrocatalysis benchmark study. Second, the workflow-authoring efficiency metrics reported in Section 6.2.1 quantify deterministic properties of the workflow representation and MCP interface, and the present Pt(111) ORR workflow was itself generated end-to-end from a single natural-language instruction, but neither constitutes a controlled human-subject usability study against alternative workflow systems, which we defer to future work. Third, while bidirectional synchronization between AI agents and the graphical editor is functionally demonstrated, large-scale latency benchmarking across distributed network environments is deferred to future systems-oriented studies.

8 Conclusions

In summary, CatGo establishes a unified, open-source platform for interactive structure construction, visualization, and AI-assisted computational workflow automation in computational chemistry and catalysis. By combining GPU-accelerated visualization, browser-native structure generation, multi-engine electronic-structure interfaces, visual DAG workflow orchestration, and LLM-driven automation within a single web-accessible environment, CatGo removes much of the fragmentation and manual overhead associated with conventional computational workflows.

The alpha-shape-based adsorption-site identification framework enables robust detection of top, bridge, and hollow sites on complex surfaces, including stepped and reconstructed systems, without relying on simplified geometric assumptions. At the workflow level, the unified DAG architecture and AI-assisted construction framework provide a flexible paradigm in which workflows can be assembled either interactively through graphical editing or automatically through natural-language interaction with coding agents. This integration substantially lowers the barrier between workflow design, execution, and scientific interpretation.

The Pt(111)ORR demonstration highlights how complete electrocatalytic workflows can be constructed transparently from structure generation to thermochemical analysis within a single executable graph representation. In parallel, the MACE-MLP reproduction of a published multi-facet Ni catalyst study demonstrates that the same workflow infrastructure can support quantitatively reproducible atomistic simulations using machine-learning interatomic potentials, while preserving consistent DAG topology and execution logic across heterogeneous computational backends.

More broadly, CatGo illustrates a transition from fragmented, script-centric computational pipelines toward

interactive, agent-driven scientific environments in which workflow generation, execution, and analysis become tightly integrated. Future developments will focus on automated convergence and uncertainty analysis, expanded support for electronic-structure and machine-learning backends, enhanced active-site discovery capabilities, community workflow template repositories, and cloud-native deployment strategies for scalable collaborative computational research.

Author contributions

G. L. contributed to conceptualization, software development, investigation, validation, and writing of the original draft. X. M. contributed to software development, investigation, validation, and writing of the original draft. L. Z. contributed to software development, investigation, validation, and writing of the original draft. J. P., J. Y. and Y. C. contributed to software development and validation. W.-L. L. contributed to conceptualization, supervision, funding acquisition, project administration, and writing through review and editing.

Conflicts of interest

There are no conflicts to declare.

Data availability

The CatGo source code is available at <https://github.com/Hello-QM/catgo-LRG>. The latest archived version and supporting workflow artifacts are available on Zenodo at <https://doi.org/10.5281/zenodo.21552845>.

Supplementary information (SI): worked examples for elemental substitution and cation intercalation, complete MCP action palette, full-resolution interface screenshots, and a 26-node MACE-MLP workflow reproducing the multi-facet Ni catalysis study of Kreitz *et al.*¹ SI DOI to be assigned by the publisher upon acceptance. See DOI: <https://doi.org/10.1039/d6dd00273k>.

Acknowledgements

The authors gratefully acknowledge support from the U.S. Department of Energy, Office of Science, Basic Energy Sciences, CPIMS Program, under award no. DE-SC0026181. The calculations were performed using the San Diego Supercomputer Center (SDSC) Expanse at UC San Diego from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program. We gratefully acknowledge Jingyi Zhang, Dongliang Zhang, Zhengyan Jiang, and Xinyi Dong for their valuable contributions to the development and improvement of CatGo. We also thank the broader CatGo community for their continued support, feedback, and contributions.

Notes and references

- 1 B. Kreitz, G. D. Wehinger, C. F. Goldsmith and T. Turek, *The J. Phys. Chem. C.*, 2021, **125**, 2984–3000.

- 2 K. Momma and F. Izumi, *J. Appl. Crystallogr.*, 2011, **44**, 1272–1276.
- 3 A. Hjorth Larsen, J. Jørgen Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Duřak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. Bjerre Jensen, J. Kermode, J. R. Kitchin, E. Leonhard Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. Bergmann Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz, O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng and K. W. Jacobsen, *J. Phys.: Condens. Matter*, 2017, **29**, 273002.
- 4 S. P. Ong, W. D. Richards, A. Jain, G. Hautier, M. Kocher, S. Cholia, D. Gunter, V. L. Chevrier, K. A. Persson and G. Ceder, *Comput. Mater. Sci.*, 2013, **68**, 314–319.
- 5 O. S. Lee, M. C. Gather and E. Zysman-Colman, *Digital Discovery*, 2024, **3**, 1695–1713.
- 6 S. Wu, X. Ma, D. Luo, L. Li, X. Shi, X. Chang, X. Lin, R. Luo, C. Pei, C. Du, Z.-J. Zhao and J. Gong, *Natl. Sci. Rev.*, 2025, **12**, nwaf169.
- 7 A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White and P. Schwaller, *Nat. Mach. Intell.*, 2024, **6**, 525–535.
- 8 D. A. Boiko, R. MacKnight, B. Kline and G. Gomes, *Nature*, 2023, **624**, 570–578.
- 9 N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, T. He, D. Milsted, M. J. McDermott, M. Gallant, E. D. Cubuk, A. Merchant, H. Kim, A. Jain, C. J. Bartel, K. Persson, Y. Zeng and G. Ceder, *Nature*, 2023, **624**, 86–91.
- 10 F. Luo, J. Zhang, Q. Wang and C. Yang, *ACS Cent. Sci.*, 2025, **11**, 511–519.
- 11 L. Li, K. Xu, R. Su, H. Gu and P. Ma, *AI Agent*, 2025, **1**, 8.
- 12 T. Yao, J. Huang, Y. Yan, Y. Yang, Z. Wang, X. Shao, Z. Gao and W. Yang, *AI Agent*, 2025, **1**, 9.
- 13 J. Liu, T. Zhu, C. Ye, Z. Fang, H. Weng and Q. Wu, *arXiv*, preprint, arXiv:2508.07035, 2025, DOI: [10.48550/arXiv.2508.07035](https://doi.org/10.48550/arXiv.2508.07035).
- 14 Z. Wang, H. Huang, H. Zhao, C. Xu, S. Zhu, J. Janssen and V. Viswanathan, *arXiv*, preprint, arXiv:2507.14267, 2025, DOI: [10.48550/arXiv.2507.14267](https://doi.org/10.48550/arXiv.2507.14267).
- 15 M. Soleymanibrojani, R. Aydin, D. Guedes-Sobrinho, A. C. Dias, M. J. Piotrowski, W. Wenzel and C. R. C. Rêgo, *arXiv*, preprint, arXiv:2512.06404, 2025, DOI: [10.48550/arXiv.2512.06404](https://doi.org/10.48550/arXiv.2512.06404).
- 16 F. Yang and J. D. Evans, *arXiv*, preprint, arXiv:2602.00185, 2026, DOI: [10.48550/arXiv.2602.00185](https://doi.org/10.48550/arXiv.2602.00185).
- 17 T. D. Pham, A. Tanikanti and M. Keçeli, *arXiv*, preprint, arXiv:2506.06363, 2025, DOI: [10.48550/arXiv.2506.06363](https://doi.org/10.48550/arXiv.2506.06363).
- 18 Y. Zou, A. H. Cheng, A. Aldossary, J. Bai, S. X. Leong, J. A. Campos-Gonzalez-Angulo, C. Choi, C. T. Ser, G. Tom, A. Wang, Z. Zhang, I. Yakavets, H. Hao, C. Crebolder, V. Bernales and A. Aspuru-Guzik, *Matter*, 2025, **8**, 102263.
- 19 J. B. Pérez-Sánchez, Y. Zou, J. A. Campos-Gonzalez-Angulo, M. Müller, I. Gustin, A. Wang, H. Hao, T. W. Ko, C. Choi, E. S. Isbrandt, M. Ghazi Vakili, H. Xu, C. Crebolder, V. Bernales and A. Aspuru-Guzik, *arXiv*, preprint, arXiv:2602.04850, 2026, DOI: [10.48550/arXiv.2602.04850](https://doi.org/10.48550/arXiv.2602.04850).
- 20 S. G. Hari Kumar, Y. Zou, A. Wang, J. Valdés-Hernández, T. W. Ko, N. Yue, O. Leng, H. Xu, C. Crebolder, A. Aspuru-Guzik and V. Bernales, *arXiv*, preprint, arXiv:2602.17886, 2026, DOI: [10.48550/arXiv.2602.17886](https://doi.org/10.48550/arXiv.2602.17886).
- 21 C. Choi, Y. Zou, M. Müller, H. Hao, Y. Kang, J. B. Pérez-Sánchez, I. Gustin, H. Xu, A. Wang, M. Ghazi Vakili, C. Crebolder, A. Aspuru-Guzik and V. Bernales, *arXiv*, preprint, arXiv:2602.04849, 2026, DOI: [10.48550/arXiv.2602.04849](https://doi.org/10.48550/arXiv.2602.04849).
- 22 J. Bai, A. Aldossary, T. Swanick, M. Müller, Y. Kang, Z. Zhang, J. W. Lee, T. W. Ko, M. Ghazi Vakili, V. Bernales and A. Aspuru-Guzik, *arXiv*, preprint, arXiv:2602.17902, 2026, DOI: [10.48550/arXiv.2602.17902](https://doi.org/10.48550/arXiv.2602.17902).
- 23 Z. Zhang, A. Yin, A. Baweja, J. Bai, I. Gustin, V. Bernales and A. Aspuru-Guzik, *arXiv*, preprint, arXiv:2604.14609, 2026, DOI: [10.48550/arXiv.2604.14609](https://doi.org/10.48550/arXiv.2604.14609).
- 24 I. Gustin, L. Mantilla Calderón, J. B. Pérez-Sánchez, J. F. Gonthier, Y. Nakamura, K. Panicker, M. Ramprasad, Z. Zhang, Y. Zou, V. Bernales and A. Aspuru-Guzik, *arXiv*, preprint, arXiv:2512.18847, 2025, DOI: [10.48550/arXiv.2512.18847](https://doi.org/10.48550/arXiv.2512.18847).
- 25 H. Chen, J. Qiu, Y. S. Tew and X. Wang, *arXiv*, preprint, arXiv:2601.13508, 2026, DOI: [10.48550/arXiv.2601.13508](https://doi.org/10.48550/arXiv.2601.13508).
- 26 J. Ock, R. S. Meda, T. Vinchurkar, Y. Jadhav and A. Barati Farimani, *arXiv*, preprint, arXiv:2410.16658, 2024, DOI: [10.48550/arXiv.2410.16658](https://doi.org/10.48550/arXiv.2410.16658).
- 27 Q. Campbell, S. Cox, J. Medina, B. Watterson and A. D. White, *arXiv*, preprint, arXiv:2502.09565, 2025, DOI: [10.48550/arXiv.2502.09565](https://doi.org/10.48550/arXiv.2502.09565).
- 28 S. Jia, C. Zhang and V. Fung, *arXiv*, preprint, arXiv:2406.13163, 2024, DOI: [10.48550/arXiv.2406.13163](https://doi.org/10.48550/arXiv.2406.13163).
- 29 R. Nduma, H. Park and A. Walsh, *arXiv*, preprint, arXiv:2512.00977, 2025, DOI: [10.48550/arXiv.2512.00977](https://doi.org/10.48550/arXiv.2512.00977).
- 30 T. Jaffrelot Inizan, S. Yang, A. Kaplan, Y.-h. Lin, J. Yin, S. Mirzaei, M. Abdelgaid, A. H. Alawadhi, K. Cho, Z. Zheng, E. D. Cubuk, C. Borgs, J. T. Chayes, K. A. Persson and O. M. Yaghi, *arXiv*, preprint, arXiv:2504.14110, 2025, DOI: [10.48550/arXiv.2504.14110](https://doi.org/10.48550/arXiv.2504.14110).
- 31 Y. Kang and J. Kim, *Nat. Commun.*, 2024, **15**, 4705.
- 32 L. Yao, S. Samantray, A. Ghosh, K. Roccapiore, L. Kovarik, S. Allec and M. Ziatdinov, *arXiv*, preprint, arXiv:2508.06569, 2025, DOI: [10.48550/arXiv.2508.06569](https://doi.org/10.48550/arXiv.2508.06569).
- 33 J. Lee, J. Ely, K. Zhang, A. Ajith, C. R. Campbell and K. Choudhary, *arXiv*, preprint, arXiv:2512.11935, 2025, DOI: [10.48550/arXiv.2512.11935](https://doi.org/10.48550/arXiv.2512.11935).
- 34 J. Hu, H. Nawaz, Y.-F. Hou, Y. Rui, L. Chi, Y. Chen, A. Ullah and P. O. Dral, *arXiv*, preprint, arXiv:2505.08195, 2025, DOI: [10.48550/arXiv.2505.08195](https://doi.org/10.48550/arXiv.2505.08195).
- 35 B. Yu, F. N. Baker, Z. Chen, G. Herb, B. Gou, D. Adu-Ampratwum, X. Ning and H. Sun, *arXiv*, preprint, arXiv:2411.07228, 2024, DOI: [10.48550/arXiv.2411.07228](https://doi.org/10.48550/arXiv.2411.07228).
- 36 X. Tang, T. Hu, M. Ye, Y. Shao, X. Yin, S. Ouyang, W. Zhou, P. Lu, Z. Zhang, Y. Zhao, A. Cohan and M. Gerstein, *arXiv*, preprint, arXiv:2501.06590, 2025, DOI: [10.48550/arXiv.2501.06590](https://doi.org/10.48550/arXiv.2501.06590).

- 37 Z. Li, B. Zhang, J. Xiao, Z. Zhou, F. Cao, J. Liang and Y. Qi, *arXiv*, preprint, arXiv:2505.21569, 2025, DOI: [10.48550/arXiv.2505.21569](https://doi.org/10.48550/arXiv.2505.21569).
- 38 H. Kim, Y. Jang and S. Ahn, *arXiv*, preprint, arXiv:2505.20820, 2025. DOI: [10.48550/arXiv.2505.20820](https://doi.org/10.48550/arXiv.2505.20820).
- 39 Z. Yang, W. Liu, B. Gao, T. Xie, Y. Li, W. Ouyang, S. Poria, E. Cambria and D. Zhou, *arXiv*, preprint, arXiv:2410.07076, 2024, DOI: [10.48550/arXiv.2410.07076](https://doi.org/10.48550/arXiv.2410.07076).
- 40 A. Jain, S. P. Ong, W. Chen, B. Medasani, X. Qu, M. Kocher, M. Brafman, G. Petretto, G.-M. Rignanese, G. Hautier, D. Gunter and K. A. Persson, *Concurrency Comput. Pract. Ex.*, 2015, **27**, 5037–5059.
- 41 G. Pizzi, A. Cepellotti, R. Sabatini, N. Marzari and B. Kozinsky, *Comput. Mater. Sci.*, 2016, **111**, 218–230.
- 42 S. P. Huber, S. Zoupanos, M. Uhrin, L. Talirz, L. Kahle, R. Häuser, N. Mounet, A. V. Yakutovich, C. W. Andersen, F. F. Ramirez, *et al.*, *Sci. Data*, 2020, **7**, 300.
- 43 A. M. Ganose, H. Sahasrabudde, M. Asta, K. Beck, T. Biswas, A. Bonkowski, J. Bustamante, X. Chen, Y. Chiang, D. C. Chrzan *et al.*, *atomate2: modular workflows for materials science*, 2025, <https://github.com/materialsproject/atomate2>.
- 44 A. S. Rosen, quacc: The Quantum Accelerator, *Software package*, 2024, <https://github.com/Quantum-Accelerators/quacc>.
- 45 J. Riebesell and M. Evans, *matterviz: visualization toolkit for materials informatics, Version 0.3.1*, 2026, <https://github.com/janosh/matterviz>.
- 46 G. Kresse and J. Hafner, *Phys. Rev. B: Condens. Matter Mater. Phys.*, 1993, **47**, 558–561.
- 47 G. Kresse and J. Furthmüller, *Comput. Mater. Sci.*, 1996, **6**, 15–50.
- 48 J. Hutter, M. Iannuzzi, F. Schiffmann and J. VandeVondele, *WIREs Comput. Mol. Sci.*, 2014, **4**, 15–25.
- 49 T. D. Kühne, *et al.*, *J. Chem. Phys.*, 2020, **152**, 194103.
- 50 F. Neese, *WIREs Comput. Mol. Sci.*, 2022, **12**, e1606.
- 51 A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott and S. J. Plimpton, *Comput. Phys. Commun.*, 2022, **271**, 108171.
- 52 P. Giannozzi, *et al.*, *J. Phys.: Condens. Matter*, 2009, **21**, 395502.
- 53 P. Giannozzi, *et al.*, *J. Phys.: Condens. Matter*, 2017, **29**, 465901.
- 54 Anthropic, Claude Code, CLI for agentic coding with Claude, Command-line coding agent, 2025, <https://docs.claude.com/en/docs/claude-code>.
- 55 OpenAI, Codex CLI, command-line interface for OpenAI coding agents, Command-line coding agent, 2025, <https://github.com/openai/codex>.
- 56 Google, Gemini CLI: command-line interface for Google Gemini coding agents, , Command-line coding agent, 2025, <https://github.com/google-gemini/gemini-cli>.
- 57 Anthropic, Model Context Protocol, Protocol specification and developer documentation, 2024, <https://modelcontextprotocol.io/>.
- 58 V. Wang, N. Xu, J.-C. Liu, G. Tang and W.-T. Geng, *Comput. Phys. Commun.*, 2021, **267**, 108033.
- 59 Jmol Development Team, Jmol, An Open-Source Java Viewer for Chemical Structures in 3D, <https://jmol.sourceforge.net/>, 2026, Accessed: 2026-03-29.
- 60 S. Maintz, V. L. Deringer, A. L. Tchougréeff and R. Dronskowski, *J. Comput. Chem.*, 2016, **37**, 1030–1035.
- 61 J. D. Hunter, *Comput. Sci. Eng.*, 2007, **9**, 90–95.
- 62 F. Neese, *WIREs Comput. Mol. Sci.*, 2012, **2**, 73–78.
- 63 M. J. Frisch, *et al.*, *Gaussian 16, Revision C.01*, Gaussian, Inc., Wallingford CT, 2016.
- 64 C. Bannwarth, S. Ehlert and S. Grimme, *J. Chem. Theory Comput.*, 2019, **15**, 1652–1671.
- 65 I. Batatia, D. P. Kovács, G. N. C. Simm, C. Ortner and G. Csányi, *Advances in Neural Information Processing Systems*, 2022, pp. 11423–11436.
- 66 B. Deng, P. Zhong, K. Jun, J. Riebesell, K. Han, C. J. Bartel and G. Ceder, *Nat. Mach. Intell.*, 2023, **5**, 1031–1041.
- 67 C. Chen and S. P. Ong, *Nat. Comput. Sci.*, 2022, **2**, 718–728.
- 68 Y. Park, J. Kim, S. Hwang and S. Han, *J. Chem. Theory Comput.*, 2024, **20**, 4857–4868.
- 69 L. Chanussot, A. Das, S. Goyal, T. Lavril, M. Shuaibi, M. Riviere, K. Tran, J. Heras-Domingo, C. Ho, W. Hu, A. Palizhati, A. Sriram, B. Wood, J. Yoon, D. Parikh, C. L. Zitnick and Z. Ulissi, *ACS Catal.*, 2021, **11**, 6059–6072.
- 70 R. Tran, J. Lan, M. Shuaibi, B. M. Wood, S. Goyal, A. Das, J. Heras-Domingo, A. Kolluru, A. Rizvi, N. Shoghi, A. Sriram, F. Therrien, J. Abed, O. Voznyy, E. H. Sargent, Z. Ulissi and C. L. Zitnick, *ACS Catal.*, 2023, **13**, 3066–3084.
- 71 A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon and E. D. Cubuk, *Nature*, 2023, **624**, 80–85.
- 72 Meta FAIR Chemistry Team, fairchem: A library for large-scale machine learning on atomic systems, Includes the Universal Model for Atoms (UMA) and the Open Catalyst models (OC20/OC22/ODAC23), 2024, <https://github.com/facebookresearch/fairchem>.
- 73 E. D. Hermes, K. Sargsyan, H. N. Najm and J. Zádor, *J. Chem. Theory Comput.*, 2022, **18**, 6974–6988.
- 74 E. Epifanovsky, *et al.*, *J. Chem. Phys.*, 2021, **155**, 084801.
- 75 G. Henkelman, A. Arnaldsson and H. Jónsson, *Comput. Mater. Sci.*, 2006, **36**, 354–360.
- 76 S. Carr, D. Massatt, S. Fang, P. Cazeaux, M. Luskin and E. Kaxiras, *Phys. Rev. B*, 2017, **95**, 075420.
- 77 shm-phy, MOIRE-LATTICE_NANO-TUBE, https://github.com/shm-phy/MOIRE-LATTICE_NANO-TUBE, 2026, Accessed: 2026-03-29.
- 78 J. L. F. Abascal and C. Vega, *J. Chem. Phys.*, 2005, **123**, 234505.
- 79 K. Shiraishi, *J. Phys. Soc. Jpn.*, 1990, **59**, 3455–3458.
- 80 A. Zur and T. C. McGill, *J. Appl. Phys.*, 1984, **55**, 378–386.
- 81 B. Delaunay, *Izv. Akad. Nauk SSSR, Otd. Mat. Estestv. Nauk.*, 1934, **7**(6), 793–800.
- 82 A. Bowyer, *Comput. J.*, 1981, **24**, 162–166.
- 83 D. F. Watson, *Comput. J.*, 1981, **24**, 167–172.
- 84 H. Edelsbrunner and E. P. Mücke, *ACM Trans. Graphics*, 1994, **13**, 43–72.

- 85 R. Dronskowski and P. E. Blöchl, *J. Phys. Chem.*, 1993, **97**, 8617–8624.
- 86 B. Hammer and J. K. Nørskov, *Adv. Catal.*, 2000, **45**, 71–129.
- 87 M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess and E. Lindahl, *SoftwareX*, 2015, **1–2**, 19–25.
- 88 D. A. Case, K. Belfon, I. Y. Ben-Shalom, S. R. Brozell, D. S. Cerutti, T. E. Cheatham, V. W. D. Cruzeiro, T. A. Darden, R. E. Duke, G. M. Giambasu, M. K. Gilson, H. Gohlke, A. W. Goetz, R. Harris, S. Izadi, S. A. Izmailov, K. Kasavajhala, M. C. Kaymak, E. King, A. Kovalenko, T. Kurtzman, T.-S. Lee, S. LeGrand, P. Li, C. Lin, J. Liu, T. Luchko, R. Luo, M. R. Machado, V. Man, K. M. Merz, Y. Miao, O. Mikhailovskii, G. Monard, H. Nguyen, K. A. O'Hearn, A. Onufriev, F. Pan, S. Pantano, R. Qi, A. Rahnamoun, D. R. Roe, A. Roitberg, C. Sagui, S. Schott-Verdugo, J. Shen, C. Simmerling, N. Skrynnikov, J. Smith, J. Swails, R. C. Walker, J. Wang, H. Wei, R. M. Wolf, X. Wu, Y. Xue, D. M. York, S. Zhao and P. A. Kollman, *J. Chem. Inf. Model.*, 2023, **63**, 6183–6191.
- 89 L. Zhang, G. Liu, X. Ma, A. Allangawi and W.-L. Li, *Chem. Sci.*, 2026, DOI: [10.1039/D6SC02460B](https://doi.org/10.1039/D6SC02460B).
- 90 O. Stern, *Z. Elektrochem. Angew. Phys. Chem.*, 1924, **30**, 508–516.
- 91 D. C. Grahame, *Chem. Rev.*, 1947, **41**, 441–501.
- 92 E. N. Baker and R. E. Hubbard, *Prog. Biophys. Mol. Biol.*, 1984, **44**, 97–179.
- 93 P. Wernet, D. Nordlund, U. Bergmann, M. Cavalleri, M. Odelius, H. Ogasawara, L.-A. Näslund, T. K. Hirsch, L. Ojamäe, P. Glatzel, L. G. M. Pettersson and A. Nilsson, *Science*, 2004, **304**, 995–999.
- 94 J. Liang, Y. Li, C.-W. Chang, M. Qiao, Z. Feng, C. Dun, W.-L. Li and G. Wu, *J. Am. Chem. Soc.*, 2026, **148**, 16337–16349.
- 95 K. Pearson, *London, Edinburgh Dublin Philos. Mag. J. Sci.*, 1901, **2**, 559–572.
- 96 L. van der Maaten and G. Hinton, *J. Mach. Learn. Res.*, 2008, **9**, 2579–2605.
- 97 L. McInnes, J. Healy and J. Melville, UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction, 2018.
- 98 K. Lum, D. Chandler and J. D. Weeks, *J. Phys. Chem. B*, 1999, **103**, 4570–4577.
- 99 D. Chandler, *Nature*, 2005, **437**, 640–647.
- 100 S. N. Jamadagni, R. Godawat and S. Garde, *Annu. Rev. Chem. Biomol. Eng.*, 2011, **2**, 147–171.
- 101 A. Serva, M. Salanne, M. Havenith and S. Pezzotti, *Proc. Natl. Acad. Sci. U. S. A.*, 2021, **118**, e2023867118.
- 102 X. Yang, G. Liu, X. Ma, X. Xiao, A. Allangawi, H. Zhang and W.-L. Li, *J. Phys. Chem. C.*, 2025, **129**, 16958–16966.
- 103 M. Nolten, K. T. Xia, S. Pezzotti, G. Schwaab, R. G. Bergman, K. N. Raymond, F. Dean Toste, T. Head-Gordon, W.-L. Li and M. Havenith, *Phys. Chem. Chem. Phys.*, 2025, **27**, 10120–10128.
- 104 F. Sebastiani, T. A. Bender, S. Pezzotti, W.-L. Li, G. Schwaab, R. G. Bergman, K. N. Raymond, F. D. Toste, T. Head-Gordon and M. Havenith, *Proc. Natl. Acad. Sci. U. S. A.*, 2020, **117**, 32954–32961.
- 105 J. Wu, Y. Li, M. Hu, T. Li, M. Yi, X. Xiao, H. Li, Y. Guo, Y. Kobayashi, M. Rueping, W.-L. Li, H. Zhang and K.-W. Huang, *Angew. Chem., Int. Ed.*, 2026, **65**, e202516163.
- 106 X. Ma, G. Liu and W.-L. Li, *Chem Catal.*, 2026, **6**, 101664.
- 107 J. K. Nørskov, J. Rossmeisl, A. Logadottir, L. Lindqvist, J. R. Kitchin, T. Bligaard and H. Jónsson, *J. Phys. Chem. B*, 2004, **108**, 17886–17892.
- 108 J. Rossmeisl, A. Logadottir and J. K. Nørskov, *Chem. Phys.*, 2005, **319**, 178–184.
- 109 J. Rossmeisl, K. Chan, R. Ahmed, V. Tripković and M. E. Björketun, *Phys. Chem. Chem. Phys.*, 2013, **15**, 10321.
- 110 J. K. Nørskov, T. Bligaard, A. Logadottir, J. R. Kitchin, J. G. Chen, S. Pandalov and U. Stimming, *J. Electrochem. Soc.*, 2005, **152**, J23–J26.
- 111 A. A. Peterson, F. Abild-Pedersen, F. Studt, J. Rossmeisl and J. K. Nørskov, *Energy Environ. Sci.*, 2010, **3**, 1311–1315.
- 112 E. Skúlason, T. Bligaard, S. Gudmundsdóttir, F. Studt, J. Rossmeisl, F. Abild-Pedersen, T. Vegge, H. Jónsson and J. K. Nørskov, *Phys. Chem. Chem. Phys.*, 2012, **14**, 1235–1245.
- 113 F. Abild-Pedersen, J. Greeley, F. Studt, J. Rossmeisl, T. R. Munter, P. G. Moses, E. Skúlason, T. Bligaard and J. K. Nørskov, *Phys. Rev. Lett.*, 2007, **99**, 016105.
- 114 G. Wulff, *Z. Kristallogr. Mineral.*, 1901, **34**, 449–530.
- 115 H. Jónsson, G. Mills and K. W. Jacobsen, *Classical and Quantum Dynamics in Condensed Phase Simulations*, World Scientific, 1998, pp. 385–404.
- 116 G. Henkelman, B. P. Uberuaga and H. Jónsson, *J. Chem. Phys.*, 2000, **113**, 9901–9904.
- 117 Anthropic, The Claude 3 Model Family: Opus, Sonnet, Haiku, *Technical report*, 2024, <https://www.anthropic.com/news/claude-3-family>.
- 118 P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel and D. Kiela, *Advances in Neural Information Processing Systems*, 2020, pp. 9459–9474.
- 119 A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 28492–28518.
- 120 Z.-D. He, S. Hanselman, Y.-X. Chen, M. T. M. Koper and F. Calle-Vallejo, *J. Phys. Chem. Lett.*, 2017, **8**, 2243–2246.